Machine Learning → Deep Learning → Computer Vision

Complete notes from your first model to your YOLO object detection project

Made for **Rabin Poudel** · ECI Engineering · alongside AI Udaan (Digital Pathshala)

> *You don't need to memorize everything. Understand the idea, memorize the yellow boxes, run every code block, and build projects.*

How to read the markers

★★★ MUST KNOW	Core idea. Asked in almost every interview and used in every project. Read it twice.
★★ IMPORTANT	You will use it often. Understand it well.
★ GOOD TO KNOW	Know what it is and when to use it. Don't go deep.
🧠 MEMORIZE	Yellow dashed boxes. Formulas and code you should know by heart.
highlighted text	Key words and rules worth remembering.

Special boxes

💡 **Tip:** Practical advice that saves you time.

⚠️ **Common mistake:** Bugs that beginners (and experts) make.

🎤 **Interview:** How the topic is asked in interviews.

🛠️ **For your project:** How it connects to your waste detection (YOLO) or intrusion detection project.

How to study each page

1. **Read** the idea and look at the diagram.
2. **Run** the code yourself. Type it, don't paste it.
3. **Break it:** change one value (learning rate, k, depth) and watch what happens.
4. **Explain** it in 3 sentences in your own words. If you can't, re-read.
5. **Remember the steps**, not the lines. Look up syntax freely.

Tools you need

- **Done** ✅ Python, NumPy, Pandas, Matplotlib
- **Part 1:** scikit-learn (+ xgboost)
- **Part 2 & 3:** PyTorch, torchvision
- **YOLO:** ultralytics
- **Free GPU:** Google Colab or Kaggle notebooks (Runtime → change runtime type → GPU)

install (terminal or Colab with !)

```
pip install scikit-learn xgboost
pip install torch torchvision ultralytics
```

Your 6-month plan

Month	Study	Build
1	Part 1: data prep, regression, gradient descent, classification, metrics	1 mini project on a Kaggle dataset
2	Part 1: trees, ensembles, clustering, pipelines + start Part 2	IDS literature review, logistic regression from scratch
3	Part 2: neural networks, PyTorch training loop	MNIST, then CIFAR-10 CNN
4	Part 3: CNN, transfer learning, object detection	Collect and label waste images
5	Part 3: YOLO in depth, detection metrics	Train, evaluate, run on Raspberry Pi
6	Part 4: interview questions, revision	Clean GitHub, demo video, apply for internships

The Roadmap

✅ Python → ✅ NumPy & Pandas → ✅ Matplotlib → Data Prep → Linear Regression → Gradient Descent ★ → Logistic Regression → Metrics ★ → Trees & Ensembles → Clustering & PCA → Pipelines → Neural Networks → Backprop ★ → PyTorch Loop ★ → CNN → Transfer Learning → Object Detection → 🎯 YOLO Project

After Part 1
clean data, train and honestly evaluate models on tables (your IDS project)

After Part 2
build and train neural networks in PyTorch, debug training

After Part 3
train CNNs, fine-tune pretrained models, train YOLO on your own images

After Part 4
present projects well and answer common interview questions

💡 **Tip:** Parts 1 to 3 build on each other. Gradient descent (Part 1) is exactly how neural networks learn (Part 2), and CNNs (Part 3) are neural networks for images. Don't skip the ★ topics.

Contents

Part 1: Machine Learning

ML-01	What is ML? Types of ML, key vocabulary ★★★	3
ML-02	The ML workflow, train/val/test split, data leakage ★★★	4
ML-03	Data preprocessing: missing values, outliers, encoding, scaling ★★★	5
ML-04	Linear Regression ★★★	6
ML-05	Gradient Descent (how models learn) ★★★	7
ML-06	Regression metrics and Polynomial Regression ★★	8
ML-07	Overfitting, bias-variance, regularization ★★★	9
ML-08	Logistic Regression (binary classification) ★★★	10
ML-09	Multi-class and multi-label classification ★★★	11
ML-10	Classification metrics: confusion matrix, precision, recall, F1, ROC ★★★	12
ML-11	Cross-validation, hyperparameter tuning, class imbalance ★★★	13
ML-12	KNN, Naive Bayes, SVM ★★	14
ML-13	Decision Trees: entropy, Gini, information gain, ID3 ★★★	15
ML-14	Ensembles: Random Forest, Boosting, XGBoost ★★★	16
ML-15	Clustering: K-Means, Hierarchical, DBSCAN ★★	17
ML-16	PCA and Association Rules (Apriori, FP-Growth) ★★	18
ML-17	Project template: sklearn Pipeline end to end ★★★	19
ML-18	Algorithm cheat sheet and feature engineering ★★	20

Part 2: Deep Learning

DL-01	Neural networks: neurons, layers, forward pass ★★★	21
DL-02	Activation functions ★★★	22
DL-03	Loss functions and backpropagation ★★★	23
DL-04	Optimizers, learning rate, batch size, initialization ★★	24
DL-05	Training problems: overfitting, dropout, batch norm, debugging ★★★	25
DL-06	PyTorch basics: tensors, autograd, nn.Module, DataLoader ★★★	26
DL-07	The PyTorch training loop (full MNIST example) ★★★	27

Part 3: CNN & Object Detection

CV-01	Images as data and why CNNs ★★	28
CV-02	Convolution, filters, padding, stride, pooling ★★★	29
CV-03	Build and train a CNN in PyTorch + data augmentation ★★★	30
CV-04	Famous CNN architectures: LeNet to ResNet, MobileNet, EfficientNet ★★	31
CV-05	Transfer learning and fine-tuning ★★★	32
CV-06	Object detection basics: boxes, IoU, NMS, anchors ★★★	33
CV-07	Detection metrics: precision, recall, AP, mAP ★★★	34
CV-08	How YOLO works ★★★	35
CV-09	Your YOLO project step by step (waste detection) ★★★	36

Part 4: Projects & Interviews

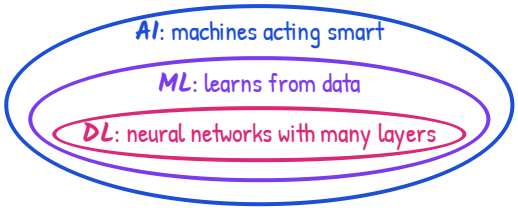
P-01	Project workflow, GitHub README, common mistakes ★★★	37
P-02	Interview questions with short answers ★★★	38
P-03	Final memorize sheet: formulas and code skeletons ★★★	39

1 What is Machine Learning?

- Machine Learning (ML) is a branch of Artificial Intelligence where computers learn patterns from data instead of following rules we write by hand.
- We give the algorithm ***examples*** (data). It finds a ***function*** that maps inputs to outputs, then uses it to make predictions on new data.
- A trained ML ***model*** is just a math function with learned numbers (parameters) inside it.

> ❝ ML = learning from examples, not from rules.

AI ⊃ ML ⊃ DL



Traditional Programming

Input + Rules (you write) → Output

Example: *if email contains "lottery" → spam.* Breaks easily.

Machine Learning

Input + Answers (labels) → Rules (model)

Give 10,000 labeled emails → model learns what spam looks like.

2 Types of Machine Learning ★★★ MUST KNOW

Supervised

Data has inputs + correct answers (labels). Model learns input → label.

- ***Regression:*** predict a number (house price, marks)
- ***Classification:*** predict a category (spam / not spam, plastic / glass)

Unsupervised

Data has no labels. Model finds hidden structure.

- ***Clustering:*** group similar customers
- ***Dimensionality reduction:*** PCA
- ***Association:*** "bread → butter"

Reinforcement

An ***agent*** takes ***actions*** in an ***environment*** and learns from rewards.

- Game AI, robots, self-driving
- Not needed for your current goals ★

Also heard of: Semi-supervised (few labels + many unlabeled) and Self-supervised (labels made from the data itself, how ChatGPT-style models are pre-trained).

3 Key Vocabulary

🧠 MEMORIZE

Feature (X)	an input column, e.g. study hours, pixel value
Label / Target (y)	the answer we want to predict
Sample / Row	one example (one student, one image)
Model	the function that maps X → y
Training	finding the best parameters using data
Inference	using the trained model on new data
Parameters	learned by the model: weights **w**, bias **b**
Hyperparameters	set by YOU before training: learning rate, k, max_depth, epochs
Loss / Cost	a number that says how wrong the model is (lower = better)
Epoch	one full pass over the training data
Generalization	doing well on new, unseen data (the real goal)

4 Regression vs Classification

	Regression	Classification
Output	number	class / category
Example	salary = 45,300	pass / fail
Models	Linear Reg., RF Regressor	Logistic Reg., RF, SVM, CNN
Metrics	MAE, RMSE, R^2	accuracy, precision, recall, F1

🎤 ***Interview:*** "What is the difference between parameters and hyperparameters?" Parameters are learned from data; hyperparameters are chosen by you and tuned using validation data.

🛠️ ***For your project:*** Waste detection = supervised (images + box labels). Intrusion detection = supervised classification (normal vs attack types).

1 The ML Project Workflow ★★★ MUST KNOW

1. Define problem → 2. Get data → 3. Explore (EDA) → 4. Clean & prepare → 5. Split data → 6. Train baseline → 7. Evaluate → 8. Improve & tune → 9. Final test → 10. Deploy & monitor

- **Define:** regression or classification? What metric matters? (e.g. for intrusion detection, missing an attack is worse than a false alarm → recall)
- **EDA:** `df.info()`, `df.describe()`, missing values, class balance, histograms, correlations.
- **Baseline:** start with a simple model (Linear / Logistic Regression). Any fancy model must beat it.
- **Improve:** better features, other models, tune hyperparameters, more data.
- **Final test:** touch the test set only once, at the very end.
- **Monitor:** real-world data changes over time ("data drift"), so models need retraining.

2 Train / Validation / Test Split ★★★ MUST KNOW

TRAIN 70% · model learns from this	VAL 15% · tune	TEST 15% · final

Train set: the model sees this and fits its parameters.

Validation set: used to pick hyperparameters and compare models. The model never trains on it.

Test set: an honest final exam. If you use it to make choices, your score becomes too optimistic.

splitting data correctly

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

X, y = load_breast_cancer(return_X_y=True)

# 1st split: 85% (train+val) and 15% test
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.15, stratify=y, random_state=42)

# 2nd split: train and val from the remaining 85%
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.1765, stratify=y_temp,
    random_state=42)          # 0.1765 * 85% ≈ 15%
print(len(X_train), len(X_val), len(X_test))
```

🧠 MEMORIZE

stratify=y keeps the same class ratio in every split (very important for imbalanced data).
random_state makes the split the same every run (reproducible).
Common splits: ***80/20*** (train/test) or ***70/15/15***.

💡 ***Tip:*** With small data, skip the fixed val set and use ***cross-validation*** on the train set instead (page ML-11).

3 Data Leakage: the #1 silent bug ★★★ MUST KNOW

Data leakage = information from the test set (or from the future) sneaks into training. The model looks amazing in your notebook, then fails in the real world.

Common causes

- Training on test data by mistake: `model.fit(X_test, y_test)`
- Fitting a scaler / imputer / encoder on the ***full*** data before splitting
- Duplicate rows that land in both train and test
- Near-identical images (same scene, same object) in train and test
- A feature that secretly contains the answer (e.g. "refund_issued" when predicting fraud)

🧠 MEMORIZE

Golden rule: split first. Then ***fit*** every preprocessing step on ***train only***, and only ***transform*** val/test.
`scaler.fit_transform(X_train)` → `scaler.transform(X_test)`

⚠️ ***Common mistake:*** 100% accuracy on a real problem almost always means leakage, a tiny test set, or a bug. Investigate before celebrating.

> "Garbage in, garbage out." Most of real ML work is preparing data.

1 Missing Values ★★ IMPORTANT

Method	When to use
Drop rows	very few rows missing (< ~5%)
Drop column	column mostly empty (> ~50-60%)
Mean	numeric, no outliers
Median	numeric with outliers (safer default)
Mode (most frequent)	categorical columns
KNN / Iterative imputer	when columns are related to each other

imputation (fit on train only!)

```python
import numpy as np
import pandas as pd
from sklearn.impute import SimpleImputer, KNNImputer

df = pd.DataFrame({"age": [20, np.nan, 25, 30],
                   "city": ["Pkr", "Ktm", np.nan, "Ktm"]})
print(df.isnull().sum())              # count missing per column

num_imp = SimpleImputer(strategy="median")
cat_imp = SimpleImputer(strategy="most_frequent")
df[["age"]] = num_imp.fit_transform(df[["age"]])
df[["city"]] = cat_imp.fit_transform(df[["city"]])
knn_imp = KNNImputer(n_neighbors=5)   # uses similar rows
```

2 Outliers ★★ IMPORTANT

🧠 MEMORIZE

IQR rule: IQR = Q3 − Q1. Outlier if x < Q1 − 1.5·IQR or x > Q3 + 1.5·IQR

Z-score rule: z = (x − mean) / std. Outlier if |z| > 3

- IQR works for skewed data. Z-score assumes roughly normal data.
- Options: remove, cap (clip to the limits), or keep. Don't blindly delete: in intrusion detection, "outliers" may be the attacks!
- Tree models are robust to outliers. Linear models, KNN and NNs are sensitive.

IQR outlier removal

```python
q1, q3 = df["age"].quantile([0.25, 0.75])
iqr = q3 - q1
low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr
df_clean = df[(df["age"] >= low) & (df["age"] <= high)]
df["age_capped"] = df["age"].clip(low, high)  # capping
```

3 Encoding Categories ★★★ MUST KNOW

Models need numbers, not text.

Encoding	Use for	Example
Ordinal / Label	categories with an **order**	low=0, medium=1, high=2
One-hot	categories with **no order**	city → is_Pkr, is_Ktm, is_Btl

⚠️ **Common mistake:** Label-encoding cities as 0, 1, 2 tells linear models that "city 2 > city 1", which is nonsense. Use one-hot. (Trees handle label encoding fine.)

encoding

```python
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder

ohe = OneHotEncoder(handle_unknown="ignore")
city_ohe = ohe.fit_transform(df[["city"]]).toarray()

ord_enc = OrdinalEncoder(categories=[["low", "medium", "high"]])
sizes = pd.DataFrame({"size": ["low", "high", "medium"]})
print(ord_enc.fit_transform(sizes))    # [[0],[2],[1]]
# quick pandas way: pd.get_dummies(df, columns=["city"])
```

EDA in a few lines: run this first on any new dataset

```python
df.info()                     # columns, types, non-nulls
print(df.describe())          # mean, std, min, max
print(df.isnull().mean())     # fraction missing per column
print(df.duplicated().sum())  # duplicate rows
num = df.select_dtypes("number")
print(num.corr().round(2))    # feature correlations
num.hist(figsize=(10, 6))     # skewed? outliers?
```

4 Feature Scaling ★★★ MUST KNOW

🧠 MEMORIZE

Standardization (StandardScaler): z = (x − μ) / σ → mean 0, std 1. Default choice.

Normalization (MinMaxScaler): x' = (x − min) / (max − min) → range [0, 1]. Used for images (pixels / 255).

Needs scaling ✅	Doesn't need it ❌
KNN, SVM, K-Means, PCA, Linear/Logistic Regression (with gradient descent or regularization), Neural Networks	Decision Trees, Random Forest, XGBoost (they split on thresholds, scale doesn't matter)

Why? Distance-based models think "salary 50,000" is far more important than "age 25" just because the number is bigger. Gradient descent also converges much faster on scaled features.

scaling the right way

```python
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)  # learn mean/std
X_test_s = scaler.transform(X_test)        # reuse them
```

What to look for ★★

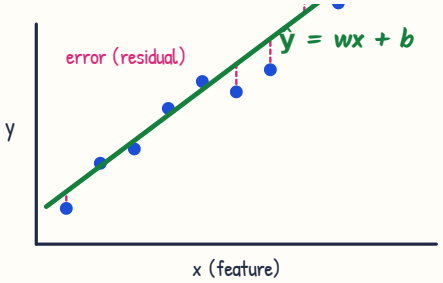
- Wrong types (numbers stored as text).
- Columns with lots of missing values.
- Skewed numbers → try `np.log1p`.
- Class balance of the target: `y.value_counts()`.
- Features almost identical to the target (leakage!).

1 The Idea ★★★ MUST KNOW

Fit the ***best straight line*** through the data to predict a number. "Best" = the line with the smallest total error.

$$\hat{y} = w \cdot x + b \quad \text{(simple: one feature)}$$

$$\hat{y} = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b \quad \text{(multiple)}$$



- **w** (weight / slope / coefficient): how much y changes when x increases by 1.
- **b** (bias / intercept): value of y when all x = 0.
- **ŷ** ("y-hat"): the model's prediction. **y**: the true value.

2 The Cost Function: MSE ★★★ MUST KNOW

🧠 MEMORIZE

$$MSE = (1/n) \cdot \Sigma (y_i - \hat{y}_i)^2$$

Average of squared errors. Squaring makes all errors positive and punishes big errors more. Training = find w and b that make MSE as small as possible.

Two ways to find the best w, b:

① ***Normal Equation*** (exact formula, one step):
$\beta = (X^TX)^{-1} X^T y$
Good for small data. Slow when features are many (matrix inverse is $O(n^3)$).

② ***Gradient Descent*** (step by step, next page). Works for huge data and is how neural networks learn.

3 Code: sklearn

linear regression with scikit-learn

```
from sklearn.datasets import load_diabetes
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_absolute_error

X, y = load_diabetes(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=2)

model = LinearRegression()          # 1. create
model.fit(X_train, y_train)         # 2. learn w and b
y_pred = model.predict(X_test)      # 3. predict

print("weights:", model.coef_)      # one w per feature
print("bias:", model.intercept_)
print("R2:", r2_score(y_test, y_pred))
print("MAE:", mean_absolute_error(y_test, y_pred))
```

normal equation from scratch (NumPy)

```
import numpy as np
Xb = np.c_[np.ones(len(X_train)), X_train]   # add column of 1s for b
beta = np.linalg.pinv(Xb.T @ Xb) @ Xb.T @ y_train   # pinv = stable inverse
b, w = beta[0], beta[1:]
y_pred_np = X_test @ w + b                   # same result as sklearn
```

4 Assumptions ★ GOOD TO KNOW

- ***Linearity:*** relationship between X and y is roughly linear.
- ***Independence:*** rows don't depend on each other.
- ***Homoscedasticity:*** errors have similar spread everywhere.
- ***Normal residuals:*** errors are roughly bell-shaped.
- ***No multicollinearity:*** features aren't strongly correlated with each other (makes weights unstable).

💡 ***Tip:*** Check by plotting residuals (y − ŷ) vs ŷ. A random cloud = good. A curve = add polynomial features.

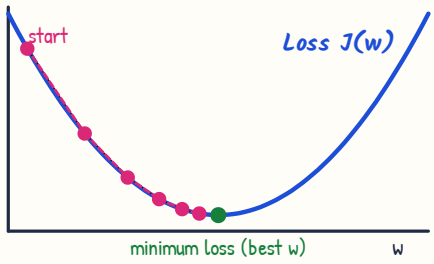
🎤 ***Interview:*** "How do you interpret a coefficient?" w = 9.2 for StudyHours means each extra hour adds about 9.2 marks, keeping other features fixed.

⚠️ ***Common mistake:*** Use `np.linalg.pinv` or `np.linalg.solve` instead of `inv`. Plain inverse fails or becomes unstable when features are correlated.

1 The Intuition ★★★ MUST KNOW

Imagine standing on a foggy hill and you want to reach the lowest valley. You can't see far, so you ***feel the slope*** under your feet and take a small step downhill. Repeat until the ground is flat.

- The hill = the ***loss function*** J(w, b).
- Your position = current ***weights***.
- The slope = the ***gradient*** (derivative). It points uphill, so we move the opposite way.
- Step size = the ***learning rate*** α.



🧠 MEMORIZE

$$w = w - \alpha \cdot \partial J/\partial w \qquad b = b - \alpha \cdot \partial J/\partial b$$

For linear regression with MSE ($\hat{y} = Xw + b$):

$$\partial J/\partial w = (2/n) \cdot X^T(\hat{y} - y)$$
$$\partial J/\partial b = (2/n) \cdot \Sigma(\hat{y} - y)$$

Same 4 steps every iteration: predict → compute loss → compute gradients → update. Neural networks do exactly this.

Learning rate α (hyperparameter)



Typical values: 0.1, 0.01, 0.001. If loss explodes or becomes NaN → lower it. If loss decreases very slowly → raise it.

2 Types of Gradient Descent ★★★ MUST KNOW

Type	Data used per update	Pros	Cons
Batch GD	ALL rows	smooth, stable	slow, needs lots of memory
Stochastic (SGD)	1 row	fast updates, can escape bad spots	very noisy
Mini-batch ✅	a batch (32, 64, 128 rows)	best of both, uses GPU well	one more hyperparameter (batch size)

Deep learning always uses ***mini-batch***. 1 epoch = one full pass over the data; with 1,000 rows and batch size 100 → 10 iterations (updates) per epoch.

3 Code: Linear Regression with Gradient Descent

from scratch with NumPy (vectorized)

```python
import numpy as np

class LinearRegressionGD:
    def __init__(self, lr=0.01, epochs=1000):
        self.lr, self.epochs = lr, epochs
        self.losses = []

    def fit(self, X, y):
        n, d = X.shape
        self.w = np.zeros(d)      # start with zeros
        self.b = 0.0
        for _ in range(self.epochs):
            y_hat = X @ self.w + self.b          # 1. predict
            loss = np.mean((y - y_hat) ** 2)     # 2. loss
            dw = (2 / n) * X.T @ (y_hat - y)     # 3. gradients
            db = (2 / n) * np.sum(y_hat - y)
            self.w -= self.lr * dw               # 4. update
            self.b -= self.lr * db
            self.losses.append(loss)
        return self

    def predict(self, X):
        return X @ self.w + self.b

# features MUST be scaled for gradient descent
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_diabetes
X, y = load_diabetes(return_X_y=True)
X = StandardScaler().fit_transform(X)
model = LinearRegressionGD(lr=0.05, epochs=2000).fit(X, y)
print(model.losses[0], model.losses[-1])  # drops a lot
```

How the code works

- `X @ self.w` multiplies every row by the weights at once (vectorized, no loops, fast).
- `y_hat - y` is the error for every row.
- `X.T @ error` gives the gradient for every weight in one line.
- `-=` moves weights downhill.
- Save losses and plot them: `plt.plot(model.losses)`. The curve must go ***down*** and flatten.

⚠️ ***Common mistake:*** Not scaling features. Gradient descent then zig-zags and needs a tiny learning rate.

🎤 ***Interview:*** "Why mini-batch over batch GD?" Faster updates, fits in GPU memory, and the small noise can help escape poor regions.

💡 ***Tip:*** Compare your weights with sklearn's `LinearRegression().coef_`. If they match, your code is right.

1 Regression Metrics ★★★ MUST KNOW

🧠 MEMORIZE

Metric	Formula	Meaning
MAE	(1/n) Σ |y − ŷ|	average error, same unit as y. Robust to outliers.
MSE	(1/n) Σ (y − ŷ)²	punishes big errors. Unit is squared. Used as loss.
RMSE	√MSE	like MAE but punishes big errors; same unit as y.
R^2	1 − SS_{res} / SS_{tot}	fraction of variance explained. 1 = perfect, 0 = as good as predicting the mean, <0 = worse than the mean.

$SS_{res} = \Sigma(y - \hat{y})^2$, $SS_{tot} = \Sigma(y - \bar{y})^2$

Which one to report?

- **RMSE or MAE** to say "how wrong in real units" (e.g. off by Rs 3,200 on average).
- **R^2** to say "how much of the pattern the model captures".
- Report at least two: R^2 alone can hide large errors.

all metrics

```
from sklearn.metrics import (mean_absolute_error,
    mean_squared_error, r2_score)
mae = mean_absolute_error(y_test, y_pred)
rmse = mean_squared_error(y_test, y_pred) ** 0.5
r2 = r2_score(y_test, y_pred)
```

2 Polynomial Regression ★★ IMPORTANT

When data is ***curved***, a straight line underfits. Trick: create new features x^2, x^3, ... and run ***normal linear regression*** on them.

$$\hat{y} = b + w_1x + w_2x^2 + w_3x^3$$

It is still "linear" regression because it is linear in the ***weights***, just not in x. With 2 features and degree 2, the new features are: x_1, x_2, x_1^2, x_1x_2, x_2^2.

- ***Degree too low*** → underfit (misses the curve).
- ***Degree too high*** → overfit (wiggles through every point, terrible on new data).
- Choose the degree with ***validation score*** or cross-validation.
- Scale features after expanding: x^3 values get huge.

⚠️ ***Common mistake:*** Predicting with raw x on a polynomial model. New data must go through the ***same*** PolynomialFeatures transform. A ***Pipeline*** does this for you automatically.

⚠️ ***Common mistake:*** Extrapolation: polynomials go wild outside the training range. Don't predict salary at 100 years of experience from data covering 1 to 10.

polynomial regression the safe way (Pipeline)

```
import numpy as np
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression

X = np.arange(1, 11).reshape(-1, 1)                # experience 1..10 years
y = np.array([20000, 25000, 28000, 29000, 29800, 30000, 31000, 31200, 31150, 31500])

model = make_pipeline(PolynomialFeatures(degree=2, include_bias=False),
                      StandardScaler(),
                      LinearRegression())
model.fit(X, y)
print(model.predict([[5.5]]))     # pass RAW x; the pipeline expands it to [x, x²]

# choose degree with validation scores
from sklearn.model_selection import cross_val_score
for d in [1, 2, 3, 5, 8]:
    m = make_pipeline(PolynomialFeatures(d, include_bias=False), StandardScaler(), LinearRegression())
    print(d, cross_val_score(m, X, y, cv=5, scoring="neg_mean_absolute_error").mean())
```

🎤 ***Interview:*** "Is polynomial regression linear or non-linear?" It models a non-linear curve, but it is a linear model in its parameters, so it is solved with ordinary linear regression.

1 Underfitting vs Overfitting ★★★ MUST KNOW



Underfitting: model is too simple. It misses the pattern even on training data.

Overfitting: model memorizes the training data, including its noise. Great on train, bad on new data.

Goal: a model that generalizes to unseen data.

Bias vs Variance

- **Bias** = error from wrong, too-simple assumptions (a line for a curve). High bias → ***underfit***.
- **Variance** = error from being too sensitive to the exact training data. High variance → ***overfit***.
- ***Trade-off:*** making a model more complex lowers bias but raises variance. We look for the sweet spot.

$$\text{Total error} \approx \text{Bias}^2 + \text{Variance} + \text{Noise}$$

🧠 MEMORIZE

Diagnose with train vs validation error:

Train error	Val error	Problem
high	high	***Underfit*** (high bias)
low	much higher	***Overfit*** (high variance)
low	low, close	***Good fit*** ✅

Fix UNDERFITTING	Fix OVERFITTING
Use a more complex model (polynomial, trees, NN)	***More training data*** (best fix) or data augmentation
Add better features	Simpler model (lower degree, smaller max_depth)
Reduce regularization (smaller α, bigger C)	***Regularization*** (L1/L2, dropout in NNs)
Train longer (more epochs)	Early stopping, cross-validation, remove noisy features

2 Regularization ★★★ MUST KNOW

Add a ***penalty for large weights*** to the loss. Large weights = the model reacts strongly to tiny input changes = overfitting. The penalty keeps the model simpler.

Ridge (L2)

$$\text{Loss} = \text{MSE} + \alpha \cdot \Sigma\, w^2$$

Shrinks all weights toward 0 but ***never exactly 0***. Good default. Handles correlated features well.

Lasso (L1)

$$\text{Loss} = \text{MSE} + \alpha \cdot \Sigma\, |w|$$

Pushes some weights to ***exactly 0*** → automatic feature selection.

Elastic Net

$$L1 + L2 \text{ mixed}$$

Mix of both (l1_ratio). Useful with many correlated features.

Ridge and Lasso

```python
from sklearn.datasets import load_diabetes
from sklearn.linear_model import Ridge, Lasso
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

X, y = load_diabetes(return_X_y=True)

ridge = make_pipeline(StandardScaler(), Ridge(alpha=1.0))
lasso = make_pipeline(StandardScaler(), Lasso(alpha=5.0))
ridge.fit(X, y); lasso.fit(X, y)

print(ridge[-1].coef_)          # all small, none exactly 0
print(lasso[-1].coef_)          # some are exactly 0.0
print("features dropped:", (lasso[-1].coef_ == 0).sum())
```

🧠 MEMORIZE

α (alpha) = regularization strength. Bigger α → simpler model (risk underfit). α = 0 → plain linear regression.
Careful: LogisticRegression and SVM use $C = 1/\alpha$.
Smaller C = stronger regularization.

⚠️ ***Common mistake:*** Always scale features before Ridge/Lasso. The penalty treats all weights equally, so features must be on the same scale.

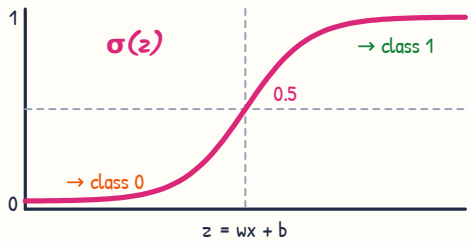
1 The Idea ★★★ MUST KNOW

Despite the name, logistic regression is a ***classification*** model. It predicts the probability that an input belongs to class 1.

1. Compute a linear score: $z = w \cdot x + b$ (any number from $-\infty$ to $+\infty$).
2. Squash it with the ***sigmoid*** into a probability between 0 and 1: $p = \sigma(z)$.
3. Decide with a threshold: $p \geq 0.5 \rightarrow$ ***class 1***, else class 0.

$$\sigma(z) = 1 / (1 + e^{-z})$$

Why not linear regression? It outputs values like 1.7 or −0.3, which aren't probabilities, and outliers drag the line badly.



$z = 0 \rightarrow p = 0.5$ (the ***decision boundary***). Large positive $z \rightarrow p$ near 1. Large negative $z \rightarrow p$ near 0.

2 Loss: Binary Cross-Entropy (Log Loss) ★★★ MUST KNOW

🧠 MEMORIZE

$$BCE = -(1/n) \Sigma \left[y \cdot \log(p) + (1-y) \cdot \log(1-p) \right]$$

$$\partial L/\partial w = (1/n) \cdot X^T (p - y) \quad \partial L/\partial b = \text{mean}(p - y)$$

Same shape as linear regression's gradient! Only the prediction changed (sigmoid added).

Intuition: if the true label is 1 and the model says $p = 0.99 \rightarrow$ loss ≈ 0.01 (tiny). If it says $p = 0.01 \rightarrow$ loss ≈ 4.6 (huge). Confident wrong answers are punished hard.

Why not MSE? With sigmoid, MSE gives a non-convex, flat loss surface. BCE is convex for logistic regression, so gradient descent finds the global minimum.

3 Code: From Scratch

logistic regression with gradient descent

```python
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

class LogisticRegressionGD:
    def __init__(self, lr=0.1, epochs=1000):
        self.lr, self.epochs = lr, epochs
        self.losses = []

    def fit(self, X, y):
        n, d = X.shape
        self.w, self.b = np.zeros(d), 0.0
        eps = 1e-9                      # avoid log(0)
        for _ in range(self.epochs):
            # 1. predict probabilities
            p = sigmoid(X @ self.w + self.b)
            # 2. loss (binary cross-entropy)
            loss = -np.mean(y * np.log(p + eps) +
                    (1 - y) * np.log(1 - p + eps))
            # 3. gradients
            dw = X.T @ (p - y) / n
            db = np.mean(p - y)
            # 4. update
            self.w -= self.lr * dw
            self.b -= self.lr * db
            self.losses.append(loss)
        return self

    def predict_proba(self, X):
        return sigmoid(X @ self.w + self.b)

    def predict(self, X, threshold=0.5):
        p = self.predict_proba(X)
        return (p >= threshold).astype(int)
```

4 Code: sklearn

compare with sklearn

```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
sc = StandardScaler().fit(X_tr)      # fit on train only
X_tr, X_te = sc.transform(X_tr), sc.transform(X_te)

mine = LogisticRegressionGD().fit(X_tr, y_tr)
sk = LogisticRegression().fit(X_tr, y_tr)
print(accuracy_score(y_te, mine.predict(X_te)))
print(accuracy_score(y_te, sk.predict(X_te)))

print(sk.predict(X_te[:3]))        # labels, e.g. [1 0 1]
# probabilities: one row per sample, [P(0), P(1)]
print(sk.predict_proba(X_te[:3]))
```

🧠 MEMORIZE

`predict()` → class labels (0/1)
`predict_proba()` → probabilities; column 1 = P(class 1)

💡 ***Tip:*** sklearn's LogisticRegression uses L2 regularization by default (C=1.0), so its weights differ slightly from yours. Plot `mine.losses`: it should fall smoothly.

1 Three Kinds of Classification ★★★ MUST KNOW

Type	Each sample has	Example	Output activation	Loss	Predict
Binary	1 of 2 classes	spam / not spam	sigmoid (1 output)	binary cross-entropy	$p \geq 0.5$
Multi-class	exactly 1 of C classes	plastic / glass / paper	**softmax** (C outputs)	categorical cross-entropy	argmax
Multi-label	any number of C labels	movie: action + comedy	**sigmoid per label**	BCE per label, averaged	each $p \geq 0.5$

2 Softmax ★★★ MUST KNOW

🧠 MEMORIZE

$$\text{softmax}(z_i) = e^{z_i} / \Sigma_j\, e^{z_j}$$

$$\text{Cross-entropy} = -\Sigma\, y_i \cdot \log(p_i) = -\log(p_{\text{true class}})$$

Turns raw scores (**logits**) into probabilities that are all positive and add up to 1. Classes compete: one goes up, others go down.

Worked example

Class	logit z	e^z	probability
plastic	2.0	7.39	**0.66** ← argmax
glass	1.0	2.72	0.24
paper	0.1	1.11	0.10
sum		11.21	1.00

If the true class is plastic: loss = $-\log(0.66)$ = **0.42**.

Multi-class strategies:
One-vs-Rest (OvR): train C binary models ("plastic vs rest", ...), pick the most confident.
Multinomial (softmax): one model, all classes together. sklearn's LogisticRegression does this by default.

3 Multi-label ★★ IMPORTANT

Labels are **independent**, so each label gets its own sigmoid. Probabilities do **not** sum to 1.

```
action: 0.91 → 1
comedy: 0.78 → 1
horror: 0.12 → 0
romance: 0.55 → 1
```

Labels are stored as a 0/1 matrix, one column per label: `[1, 1, 0, 1]`. Think of it as C binary problems solved together.

multi-class and multi-label in sklearn

```python
from sklearn.datasets import load_iris
from sklearn.datasets import make_multilabel_classification
from sklearn.linear_model import LogisticRegression
from sklearn.multioutput import MultiOutputClassifier
from sklearn.preprocessing import MultiLabelBinarizer

# multi-class (3 iris species), softmax inside
X, y = load_iris(return_X_y=True)
clf = LogisticRegression(max_iter=1000).fit(X, y)
print(clf.predict_proba(X[:1]))   # 3 probs, sum = 1

# multi-label: turn tag lists into a 0/1 matrix
mlb = MultiLabelBinarizer()
Y = mlb.fit_transform([["action", "comedy"],
                       ["horror"], ["action"]])
print(mlb.classes_, Y)

# one binary classifier per label
Xm, Ym = make_multilabel_classification(
    n_samples=300, n_classes=4, random_state=0)
multi = MultiOutputClassifier(LogisticRegression(max_iter=1000))
multi.fit(Xm, Ym)
print(multi.predict(Xm[:2]))      # e.g. [[0 1 1 0], ...]
```

Multi-label metrics: hamming loss (fraction of wrong labels), **micro / macro F1**. Plain accuracy is too strict (all labels must match).

🛠️ **For your project:** Waste classification of a **single item per image** = multi-class (softmax). YOLO detection handles many items per image by predicting a separate box + class for each object.

turn text class names into integer labels

```python
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
y_idx = le.fit_transform(["glass", "paper",
                          "plastic", "glass"])
print(y_idx)                      # [0 1 2 0]
print(le.classes_)                # ['glass' 'paper' 'plastic']
print(le.inverse_transform([2]))  # ['plastic']
```

💡 **Tip:** Use **LabelEncoder only for the target y** (class names → 0..C−1, which PyTorch's CrossEntropyLoss needs). For input features, use OneHotEncoder / OrdinalEncoder (ML-03). Save `le.classes_` so you can turn predictions back into names.

1 Confusion Matrix ★★★ MUST KNOW

	Predicted 0	Predicted 1
Actual 0	**TN** correctly said no	**FP** false alarm (Type I error)
Actual 1	**FN** missed it (Type II error)	**TP** correctly found

sklearn layout: rows = actual, columns = predicted → [[TN, FP], [FN, TP]]

🧠 MEMORIZE

- **Accuracy** (TP + TN) / total → overall correctness
- **Precision** TP / (TP + FP) → "of everything I flagged, how much was right?"
- **Recall** TP / (TP + FN) → "of all real positives, how many did I catch?" (= sensitivity, TPR)
- **F1 score** 2·P·R / (P + R) → harmonic mean of precision and recall
- **Specificity** TN / (TN + FP) → how well negatives are recognized
- **FPR** FP / (FP + TN) = 1 − specificity

Worked example: 100 emails

TP = 40, FP = 10, FN = 5, TN = 45 → ***Accuracy*** = 85/100 = 0.85 · ***Precision*** = 40/50 = 0.80 · ***Recall*** = 40/45 = 0.89 · ***F1*** = 2(0.80)(0.89)/(0.80+0.89) = 0.84

2 Which Metric When? ★★★ MUST KNOW

Precision matters

when ***false positives are costly***.
Spam filter (don't hide a real email), recommending a risky medicine.

Recall matters

when ***missing a positive is costly***.
Cancer screening, fraud, ***intrusion detection*** (a missed attack is a disaster).

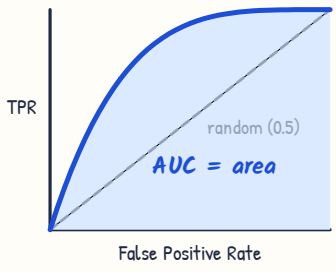
F1 matters

when you need a ***balance*** and classes are imbalanced.
Most real projects report F1.

⚠️ ***Common mistake:*** ***Accuracy paradox:*** if 99% of network traffic is normal, a model that always says "normal" gets 99% accuracy and catches ***zero*** attacks. On imbalanced data, never trust accuracy alone.

Threshold trade-off: raising the threshold (e.g. 0.5 → 0.8) → fewer positives predicted → precision ↑, recall ↓. Lowering it does the opposite. Choose the threshold based on what errors cost.

3 ROC & AUC ★★ IMPORTANT



ROC curve = TPR vs FPR at every threshold. ***AUC*** = area under it: 1.0 perfect, 0.5 random guessing. It measures how well the model ***ranks*** positives above negatives.

For heavy imbalance, also check the ***Precision-Recall curve*** and ***average precision***.

🧠 MEMORIZE

Multi-class averaging (in classification_report):
macro = simple average over classes → treats rare classes equally (best to spot weak classes)
weighted = average weighted by class size
micro = from total TP/FP/FN counts (equals accuracy for single-label multi-class)

evaluate a classifier

```
from sklearn.metrics import (confusion_matrix, classification_report,
    roc_auc_score, ConfusionMatrixDisplay, f1_score)

y_pred = sk.predict(X_te)                  # model from page ML-08
print(confusion_matrix(y_te, y_pred))
print(classification_report(y_te, y_pred, digits=3))
print("macro F1:", f1_score(y_te, y_pred, average="macro"))

y_prob = sk.predict_proba(X_te)[:, 1]      # P(class 1)
print("ROC-AUC:", roc_auc_score(y_te, y_prob))

y_pred_08 = (y_prob >= 0.8).astype(int)    # custom threshold
ConfusionMatrixDisplay.from_predictions(y_te, y_pred)  # plot
```

1 K-Fold Cross-Validation ★★★ MUST KNOW

	F1	F2	F3	F4	F5
Round 1	val	train	train	train	train
Round 2	train	val	train	train	train
Round 3	train	train	val	train	train
Round 4	train	train	train	val	train
Round 5	train	train	train	train	val

Final score = ***mean ± std*** of the 5 scores

- Split training data into ***k*** parts (folds), usually ***k = 5 or 10***.
- Train k times; each time a different fold is the validation set.
- Every row is used for validation exactly once → a more reliable score than one split, especially on small data.
- ***Stratified K-Fold:*** keeps class ratios in each fold. Use it for classification (sklearn does this by default for classifiers).
- The ***test set stays outside*** CV, untouched.
- Time series: use `TimeSeriesSplit` (never train on the future).

cross-validation

```python
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_breast_cancer

X, y = load_breast_cancer(return_X_y=True)
model = make_pipeline(StandardScaler(), LogisticRegression())  # scaler refit inside each fold (no leakage)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=cv, scoring="f1")
print(f"F1 = {scores.mean():.3f} ± {scores.std():.3f}")
```

2 Hyperparameter Tuning ★★ IMPORTANT

GridSearchCV: tries ***every*** combination. Thorough but slow (3 × 4 × 3 = 36 combos × 5 folds = 180 fits).

RandomizedSearchCV: tries ***n random*** combinations. Much faster, usually nearly as good. Use it for big search spaces.

Rule: tune with CV on training data → pick best → evaluate ***once*** on the test set.

grid search

```python
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

params = {"n_estimators": [100, 300],
          "max_depth": [None, 5, 10],
          "min_samples_leaf": [1, 3]}
grid = GridSearchCV(RandomForestClassifier(random_state=42),
                    params, cv=5, scoring="f1_macro", n_jobs=-1)
grid.fit(X, y)                 # use X_train, y_train in projects
print(grid.best_params_, round(grid.best_score_, 3))
best_model = grid.best_estimator_   # already refit on all data
```

3 Imbalanced Data ★★★ MUST KNOW

🧠 MEMORIZE

Toolbox (try in this order):

1. ***Right metrics:*** recall, F1, macro-F1, PR-AUC. Not accuracy.
2. ***stratify=y*** in splits, StratifiedKFold in CV.
3. ***class_weight="balanced"***: errors on rare classes cost more.
4. ***Threshold tuning:*** lower the threshold to catch more positives.
5. ***Resampling:*** oversample minority (SMOTE) or undersample majority, on training data only.
6. Collect more minority-class data (best, if possible).

class weights and SMOTE

```python
from sklearn.linear_model import LogisticRegression
from imblearn.pipeline import make_pipeline as imb_pipe
from imblearn.over_sampling import SMOTE
from sklearn.preprocessing import StandardScaler

clf = LogisticRegression(class_weight="balanced")

# SMOTE inside an imblearn pipeline: it only
# resamples the training folds, never validation
smote_model = imb_pipe(StandardScaler(),
                       SMOTE(random_state=42),
                       LogisticRegression(max_iter=1000))
scores = cross_val_score(smote_model, X, y, cv=5,
                         scoring="f1")
```

⚠️ ***Common mistake:*** Oversampling ***before*** splitting. Copies of the same minority row end up in train and test → leakage.

🛠️ ***For your project:*** IDS datasets are heavily imbalanced (rare attack types). Report per-class recall and macro-F1, and repeat runs with different seeds.

1 K-Nearest Neighbors ★★ IMPORTANT

1. Pick **k** (e.g. 5).
2. For a new point, compute the **distance** to every training point.
3. Take the **k closest**.
4. Classification: **majority vote**. Regression: **average**.

🧠 MEMORIZE

Euclidean: $d = \sqrt{\Sigma(x_i - y_i)^2}$ · **Manhattan:** $d = \Sigma|x_i - y_i|$

- **Lazy learner:** no real training; all work happens at prediction (slow on big data).
- Must scale features (distance-based).
- Small k → overfit (noisy). Large k → underfit (too smooth). Use odd k for binary to avoid ties; pick k with CV.
- Suffers in high dimensions ("curse of dimensionality").

KNN: choose k with CV

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score

for k in [1, 3, 5, 7, 11, 15]:
    knn = make_pipeline(StandardScaler(),
                        KNeighborsClassifier(n_neighbors=k))
    print(k, cross_val_score(knn, X, y, cv=5).mean())
```

2 Naive Bayes ★★ IMPORTANT

🧠 MEMORIZE

$$P(class \mid x) = P(x \mid class) \cdot P(class) \,/\, P(x)$$

Naive assumption: features are independent given the class → $P(class \mid x) \propto P(class) \cdot \prod P(x_i \mid class)$

- Pick the class with the highest posterior probability.
- **Very fast**, works with little data, great baseline for **text** (spam filters).
- The independence assumption is usually false, but it still works surprisingly well.

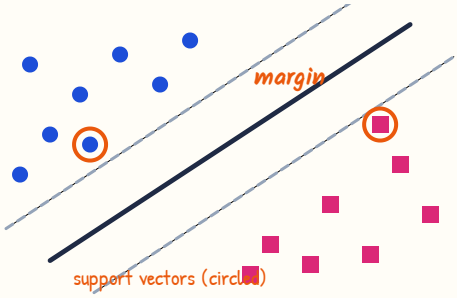
Type	Features
GaussianNB	continuous numbers (assumes normal)
MultinomialNB	word counts (text)
BernoulliNB	binary yes/no features

Naive Bayes spam filter

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import make_pipeline

texts = ["win lottery now", "meeting at 5",
         "free prize win", "project report due"]
labels = [1, 0, 1, 0]                       # 1 = spam
spam = make_pipeline(CountVectorizer(), MultinomialNB())
spam.fit(texts, labels)
print(spam.predict(["free lottery prize"]))   # [1]
```

3 Support Vector Machine (SVM) ★★ IMPORTANT



- Finds the boundary (hyperplane) with the largest margin between classes. Wide margin = better generalization.
- **Support vectors:** the points closest to the boundary. Only they decide where it goes.
- **C** (soft margin): small C → wider margin, allows some mistakes (more regularized). Large C → tries to classify every point correctly (may overfit).
- **Kernel trick:** maps data into a higher dimension where it becomes separable, without computing that space directly. Kernels: **linear**, **poly**, **RBF** (default, most flexible).
- **gamma** (RBF): how far one point's influence reaches. High gamma → wiggly boundary (overfit).
- Needs scaling. Strong on small/medium, high-dimensional data. Slow on very large datasets (> ~100k rows).

SVM with tuning

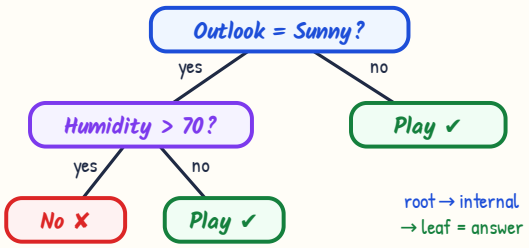
```
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV
svm = make_pipeline(StandardScaler(), SVC(kernel="rbf", probability=True))
grid = GridSearchCV(svm, {"svc__C": [0.1, 1, 10], "svc__gamma": ["scale", 0.01, 0.1]}, cv=5)
grid.fit(X, y)          # step name "svc" + "__" + parameter name
print(grid.best_params_)
```

🎤 **Interview:** "KNN vs SVM vs Naive Bayes?" NB: fastest, text, tiny data. KNN: simple, small data, needs scaling, slow predictions. SVM: strong margins on medium data, needs scaling and tuning (C, gamma).

1 How a Decision Tree Works ★★★ MUST KNOW

A tree asks a series of **yes/no questions** about the features until it reaches an answer, like a flowchart.

1. At a node, try every feature and every possible split point.
2. Pick the split that makes the child groups most pure (mostly one class).
3. Repeat on each child (recursion).
4. Stop when a node is pure, or a limit is hit (max_depth, min_samples). That node becomes a **leaf** with the majority class (or mean value for regression).



2 Measuring Purity ★★★ MUST KNOW

MEMORIZE

$$\text{Entropy } H = -\sum p_i \cdot \log_2(p_i)$$

$$\text{Gini} = 1 - \sum p_i^2$$

$$\text{Information Gain} = H(\text{parent}) - \sum (n_k/n) \cdot H(\text{child}_k)$$

p_i = fraction of class i in the node. Pure node → entropy 0, Gini 0. 50/50 split (2 classes) → entropy 1, Gini 0.5. Choose the split with the highest information gain (or the biggest drop in Gini).

Worked example

14 days: 9 Play, 5 Don't play.
$H = -(9/14)\log_2(9/14) - (5/14)\log_2(5/14) = 0.940$
$\text{Gini} = 1 - (0.643^2 + 0.357^2) = 0.459$

Split on **Outlook**: Sunny (2 Y, 3 N) $H = 0.971$ · Overcast (4 Y, 0 N) $H = 0$ · Rain (3 Y, 2 N) $H = 0.971$
Weighted $= 5/14 \cdot 0.971 + 4/14 \cdot 0 + 5/14 \cdot 0.971 = 0.694$
$IG = 0.940 - 0.694 \approx 0.247$

Algorithm	Split criterion	Notes
ID3	entropy + information gain	categorical features, multi-way splits, no pruning
C4.5	gain ratio	fixes ID3's bias toward many-valued features; handles numbers
CART ✅	Gini (or MSE for regression)	binary splits; what **sklearn** uses

entropy and information gain from scratch

```
import numpy as np

def entropy(y):
    _, counts = np.unique(y, return_counts=True)
    p = counts / counts.sum()
    return -np.sum(p * np.log2(p))

def information_gain(y, feature_column):
    parent = entropy(y)
    values, counts = np.unique(feature_column, return_counts=True)
    weighted = sum((c / len(y)) * entropy(y[feature_column == v])
                   for v, c in zip(values, counts))
    return parent - weighted

# the 14-day example: 9 Yes, 5 No
y = np.array(["Y"] * 9 + ["N"] * 5)
outlook = np.array(["S","S","O","O","O","O","R","R","R",
                    "S","S","S","R","R"])
print(round(entropy(y), 3))                    # 0.94
print(round(information_gain(y, outlook), 3))  # 0.247
# ID3 computes this for every feature and splits on the max
```

Stop overfitting

An unlimited tree grows until every leaf is pure = memorizes training data. Limit it:

- `max_depth` (e.g. 3 to 10)
- `min_samples_split`, `min_samples_leaf`
- `max_features`
- `ccp_alpha` : cost-complexity **pruning** (cut weak branches after growing)

Pros / Cons

✓ easy to explain, no scaling needed, handles non-linear patterns and mixed data types
✗ overfits easily, unstable (small data change → different tree). Fix: **ensembles** (next page).

sklearn

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree, export_text
from sklearn.datasets import load_iris
X, y = load_iris(return_X_y=True)
tree = DecisionTreeClassifier(criterion="gini",
           max_depth=3, random_state=42).fit(X, y)
print(export_text(tree))         # rules as text
print(tree.feature_importances_)
plot_tree(tree, filled=True)     # draw it
```

> *One tree is unstable. Many trees voting together are strong. ("Wisdom of the crowd")*

1 Bagging vs Boosting ★★★ MUST KNOW

	Bagging (e.g. Random Forest)	Boosting (e.g. XGBoost)
How	train many models ***in parallel***, each on a random bootstrap sample (rows drawn with replacement)	train models ***one after another***; each new model focuses on the ***mistakes*** of the previous ones
Combine	majority vote / average	weighted sum of all models
Mainly reduces	variance (overfitting)	bias (underfitting), and variance with tuning
Base model	deep trees	shallow trees ("weak learners")
Overfit risk	low, hard to overfit by adding trees	higher; needs learning_rate, early stopping

Random Forest ★★★

1. Draw a bootstrap sample of rows for each tree.
2. At ***each split***, consider only a ***random subset of features*** (√n_features for classification). This makes trees different from each other.
3. Grow many deep trees (n_estimators = 100 to 500).
4. Predict by voting (classification) or averaging (regression).

OOB score: each tree skips ~37% of rows; testing on those gives a free validation score.

Boosting family

- ***AdaBoost:*** increases the weight of misclassified samples so the next model focuses on them.
- ***Gradient Boosting:*** each new tree predicts the ***remaining errors*** (residuals / negative gradient) of the current ensemble. Prediction = sum of all trees × learning_rate.
- ***XGBoost / LightGBM / CatBoost:*** fast, regularized gradient boosting. Often the best models for tabular data and Kaggle winners. CatBoost handles categories well.

🧠 MEMORIZE

Key boosting hyperparameters: `n_estimators` (number of trees) ↔ `learning_rate` (smaller rate needs more trees, usually generalizes better) · `max_depth` (3 to 8) · `subsample`, `colsample_bytree` (randomness, like RF) · use ***early stopping*** on a validation set.

Random Forest

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

rf = RandomForestClassifier(n_estimators=300,
        max_features="sqrt", oob_score=True,
        class_weight="balanced", n_jobs=-1,
        random_state=42)
rf.fit(X_tr, y_tr)
print("OOB:", rf.oob_score_)
print("F1:", f1_score(y_te, rf.predict(X_te)))
top = rf.feature_importances_.argsort()[::-1][:5]
print("top features:", top)
```

XGBoost with early stopping

```
from xgboost import XGBClassifier

X_tr2, X_val, y_tr2, y_val = train_test_split(
    X_tr, y_tr, test_size=0.2, stratify=y_tr,
    random_state=42)
xgb = XGBClassifier(n_estimators=1000,
        learning_rate=0.05, max_depth=4,
        subsample=0.8, colsample_bytree=0.8,
        early_stopping_rounds=50,
        eval_metric="logloss", random_state=42)
xgb.fit(X_tr2, y_tr2, eval_set=[(X_val, y_val)],
        verbose=False)
print("best round:", xgb.best_iteration)
print("F1:", f1_score(y_te, xgb.predict(X_te)))
```

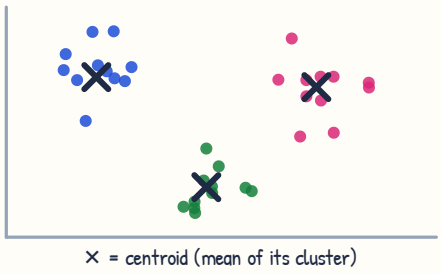
💡 ***Tip:*** sklearn's `HistGradientBoostingClassifier` is fast and also built in.

🛠️ ***For your project:*** XGBoost, LightGBM and HistGradientBoosting handle missing values (NaN) natively. In your IDS study, compare them against "impute + model" pipelines.

🎤 ***Interview:*** "Why does Random Forest overfit less than one tree?" Averaging many decorrelated trees cancels out their individual errors, reducing variance.

1 K-Means ★★★ MUST KNOW

1. Choose **k**, the number of clusters.
2. Place k ***centroids*** (K-Means++ spreads them out smartly; sklearn default).
3. ***Assign*** each point to its nearest centroid.
4. ***Update*** each centroid to the mean of its points.
5. Repeat 3 to 4 until assignments stop changing.



× = centroid (mean of its cluster)

🧠 MEMORIZE

Inertia (WCSS) = Σ (distance of each point to its centroid)² → K-Means minimizes this.

Silhouette score s = (b − a) / max(a, b), where a = mean distance to own cluster, b = mean distance to nearest other cluster. Range −1 to 1; higher = better separated clusters.



Elbow method: plot inertia vs k; pick the bend where adding clusters stops helping much.

Limitations

- You must choose k.
- Assumes round, similar-size clusters.
- Sensitive to scale (***standardize first***) and to outliers.
- Results depend on initialization; `n_init` runs it several times and keeps the best.

K-Means with elbow + silhouette

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import make_blobs

X, _ = make_blobs(n_samples=300, centers=3, random_state=0)
X = StandardScaler().fit_transform(X)
for k in range(2, 7):
    km = KMeans(n_clusters=k, n_init=10, random_state=0).fit(X)
    print(k, round(km.inertia_, 1),
          round(silhouette_score(X, km.labels_), 3))
km = KMeans(n_clusters=3, n_init=10, random_state=0).fit(X)
labels, centers = km.labels_, km.cluster_centers_
```

2 Other Clustering Methods ★★ IMPORTANT

Hierarchical (Agglomerative)

- Start: every point is its own cluster. Repeatedly ***merge the two closest clusters*** (bottom-up).
- Result is a tree called a ***dendrogram***; cut it at a height to get clusters.
- ***Linkage*** = how cluster distance is measured: single (closest points), complete (farthest), average, ***ward*** (minimizes variance; good default).
- No need to pick k in advance. Slow for big data ($O(n^2)$ memory).

DBSCAN (density-based)

- ***eps*** = neighborhood radius. ***min_samples*** = points needed to form a dense region.
- ***Core*** points (dense), ***border*** points (near a core), ***noise*** points (label −1).
- Finds clusters of any shape, no k needed, and flags outliers automatically.
- Struggles when clusters have very different densities.

hierarchical and DBSCAN

```
from sklearn.cluster import AgglomerativeClustering, DBSCAN
agg_labels = AgglomerativeClustering(n_clusters=3, linkage="ward").fit_predict(X)
db_labels = DBSCAN(eps=0.3, min_samples=5).fit_predict(X)
print("noise points:", (db_labels == -1).sum())          # -1 = outlier
# dendrogram: from scipy.cluster.hierarchy import linkage, dendrogram; dendrogram(linkage(X, "ward"))
```

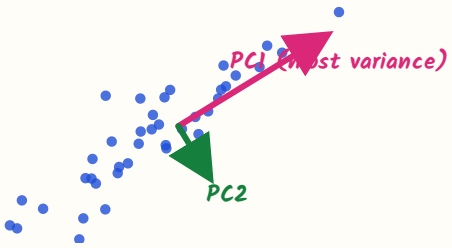
🎤 ***Interview:*** "How do you choose k?" Elbow method on inertia, silhouette score, and domain knowledge (e.g. the business wants 4 customer segments).

💡 ***Tip:*** Clustering has no labels, so evaluation is harder. Always look at the clusters (plot them, inspect examples) to check they make sense.

1 PCA: Principal Component Analysis ★★ IMPORTANT

PCA finds **new axes** (principal components) that capture the most variance (spread) in the data, then keeps only the top few. It compresses many features into fewer while keeping most information.

1. **Standardize** features (PCA is scale-sensitive).
2. Compute the **covariance matrix**.
3. Find its **eigenvectors** (directions = components) and **eigenvalues** (variance along each).
4. Sort by eigenvalue; keep the top k components.
5. **Project** the data onto them.



- **Explained variance ratio:** share of total variance each component keeps. Common rule: keep enough components for 95% variance.
- Components are **uncorrelated** (orthogonal), which also fixes multicollinearity.
- **Uses:** 2D/3D visualization, speeding up training, removing noise.
- **Cost:** new features are mixes of old ones, so they are hard to interpret. Unsupervised: ignores y.
- For visualization only, **t-SNE** and **UMAP** often show clusters better ★

PCA

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_breast_cancer

X, y = load_breast_cancer(return_X_y=True)  # 30 features
X_s = StandardScaler().fit_transform(X)

pca = PCA(n_components=0.95)   # keep 95% variance
X_p = pca.fit_transform(X_s)
print(X_p.shape)               # (569, 10): 30 → 10
print(pca.explained_variance_ratio_.round(3))

X_2d = PCA(n_components=2).fit_transform(X_s)  # to plot
```

⚠ **Common mistake:** Fit PCA on training data only, then `transform` the test data (put it in a Pipeline).

2 Association Rules ★ GOOD TO KNOW

Find items that often appear together in transactions: **{bread} → {butter}**. Used in shopping baskets, recommendations.

🧠 MEMORIZE

Support(A)	transactions containing A / total → how common
Confidence(A→B)	support(A ∪ B) / support(A) → how often B appears when A does
Lift(A→B)	confidence(A→B) / support(B) → > 1 means A and B occur together more than by chance

Example: 100 baskets

bread in 40, butter in 25, both in 20.
Support = 20/100 = **0.20**
Confidence = 20/40 = **0.50**
Lift = 0.50 / 0.25 = **2.0** → strong positive link

Apriori: builds itemsets level by level (size 1, 2, 3, …) and prunes with the rule "if an itemset is infrequent, all its supersets are infrequent". Scans the data many times; slow on big data.

FP-Growth: compresses transactions into an **FP-tree** (2 scans) and mines patterns from it **without generating candidates**. Much faster; same results.

FP-Growth with mlxtend

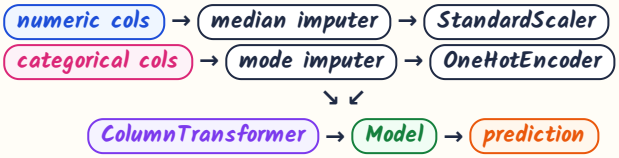
```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import fpgrowth, association_rules

baskets = [["bread", "butter", "milk"], ["bread", "butter"],
           ["milk", "eggs"], ["bread", "eggs"],
           ["bread", "butter", "eggs"]]
te = TransactionEncoder()
df = pd.DataFrame(te.fit_transform(baskets), columns=te.columns_)

freq = fpgrowth(df, min_support=0.4, use_colnames=True)
rules = association_rules(freq, metric="lift", min_threshold=1.0)
print(rules[["antecedents", "consequents",
             "support", "confidence", "lift"]])
```

1 Why Pipelines? ★★★ MUST KNOW

- A **Pipeline** chains preprocessing + model into **one object**: `fit` runs every step on training data, `predict` applies the exact same steps to new data.
- It prevents data leakage during cross-validation: the scaler/imputer is refit inside every fold.
- **ColumnTransformer** applies different steps to numeric and categorical columns.
- You save **one file** and it contains everything needed to predict.



2 The Template (reuse in every tabular project) 🧠 MEMORIZE THE STEPS

ml_template.py: replace the file name and column names with your own

```python
import pandas as pd, joblib
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix

# 1. Load and look
df = pd.read_csv("data.csv")
df = df.drop_duplicates()                       # duplicates can leak across splits
X = df.drop(columns=["target"])
y = df["target"]
print(y.value_counts(normalize=True))           # check class balance

# 2. Split FIRST (test set is locked away until the end)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

# 3. Preprocessing per column type
num_cols = X.select_dtypes(include="number").columns
cat_cols = X.select_dtypes(exclude="number").columns
preprocess = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")),
                      ("scale", StandardScaler())]), num_cols),
    ("cat", Pipeline([("impute", SimpleImputer(strategy="most_frequent")),
                      ("onehot", OneHotEncoder(handle_unknown="ignore"))]), cat_cols),
])

# 4. Full pipeline = preprocessing + model
pipe = Pipeline([("prep", preprocess),
                 ("model", RandomForestClassifier(class_weight="balanced", random_state=42))])

# 5. Tune with cross-validation on TRAIN only
params = {"model__n_estimators": [200, 400], "model__max_depth": [None, 10, 20]}
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
search = GridSearchCV(pipe, params, cv=cv, scoring="f1_macro", n_jobs=-1)
search.fit(X_train, y_train)
print("best:", search.best_params_, "CV F1:", round(search.best_score_, 3))

# 6. Final, one-time evaluation on TEST
best = search.best_estimator_
y_pred = best.predict(X_test)
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred, digits=3))

# 7. Save and reuse
joblib.dump(best, "model.joblib")
loaded = joblib.load("model.joblib")
print(loaded.predict(X_test.head(3)))           # raw rows in, predictions out
```

🧠 MEMORIZE

Parameter names in a pipeline: `stepname__param` (double underscore), e.g. `model__max_depth`, `prep__num__impute__strategy`.

💡 **Tip:** Start with a baseline (LogisticRegression) in the same pipeline, then swap in RandomForest / XGBoost. Only the "model" step changes.

🛠️ **For your project:** For your IDS missing-data study: put each imputer inside this pipeline so it is fit on training folds only.

1 Which Algorithm When? ★★★ MUST KNOW

Algorithm	Task	Scale?	Strengths	Weaknesses	Use when
Linear Regression	R	for GD/reg.	fast, interpretable	only linear patterns	baseline for numbers
Logistic Regression	C	yes	fast, probabilities, interpretable	linear boundary	baseline for classes
KNN	C/R	yes	simple, no training	slow predictions, high-dim	small data
Naive Bayes	C	no	very fast, small data	independence assumption	text, spam
SVM	C/R	yes	strong margins, kernels	slow on big data, tuning	small/medium, many features
Decision Tree	C/R	no	explainable rules	overfits, unstable	need to explain decisions
Random Forest	C/R	no	robust, little tuning	bigger, slower, less interpretable	strong default for tabular
XGBoost / LightGBM	C/R	no	top accuracy on tabular, handles NaN	more tuning	best performance on tables
K-Means / DBSCAN	clustering	yes	finds groups	k / eps choice	segmentation, outliers
Neural Networks	C/R	yes	learn complex patterns	need lots of data, compute	**images, audio, text**

🧠 MEMORIZE

Default game plan for tabular data: simple baseline (Linear/Logistic) → Random Forest → XGBoost/LightGBM → tune the best. **For images:** CNN / transfer learning / YOLO (Part 3). **For text:** TF-IDF + Logistic Regression baseline, then transformers.

🎤 **Interview:** "Why not always use deep learning?" On small/medium tabular data, gradient-boosted trees usually win, train faster, need less tuning and are easier to explain.

2 Feature Engineering ★★ IMPORTANT

Better features often beat a better model. Ideas that work again and again:

- **Ratios / differences:** price per sq.ft, bytes per packet, income − expenses.
- **Dates:** extract hour, weekday, month, is_weekend.
- **Binning:** age → age groups.
- **Log transform** skewed numbers (income, packet counts): `np.log1p(x)`.
- **Interaction features:** $x_1 \cdot x_2$ (or PolynomialFeatures).
- **Text:** word counts / TF-IDF.
- **Aggregates:** per-user average, counts per group (compute on train only).
- **Feature selection:** drop useless or highly correlated features; use feature_importances_, Lasso, or `SelectKBest`.

3 The sklearn API Pattern 🧠 MEMORIZE

Method	What it does	Used on
`fit(X, y)`	learn from data	models & transformers
`predict(X)`	labels / values	models
`predict_proba(X)`	class probabilities	classifiers
`transform(X)`	apply learned change	scalers, encoders, PCA
`fit_transform(X)`	fit + transform (train only!)	transformers
`score(X, y)`	accuracy (clf) / R^2 (reg)	models

Every model, same 4 lines

the universal pattern

```
model = AnyModel(hyperparameters)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print(metric(y_test, y_pred))
```

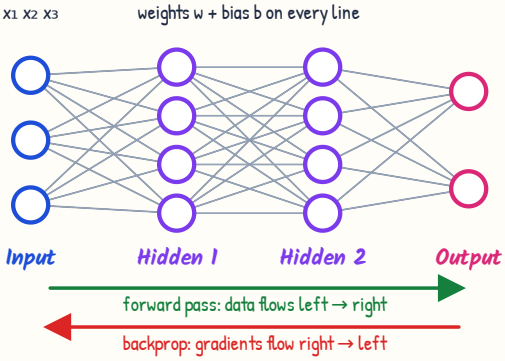
💡 **Tip:** Learned attributes end with an underscore: `coef_`, `intercept_`, `feature_importances_`, `cluster_centers_`, `best_params_`.

❝ ✅ *Part I done. You now know enough classical ML to build projects. Next: neural networks.*

1 From Logistic Regression to a Neuron

★★★ MUST KNOW

You already know a neuron! Logistic regression = ***one neuron*** with a sigmoid. A neural network is many of these, stacked in layers.



🧠 MEMORIZE

$$z = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b = w \cdot x + b$$

$$a = f(z)$$ f = activation function (ReLU, sigmoid, …)

Each neuron: weighted sum → add bias → apply activation. Output a goes to the next layer.

- ***Input layer:*** one value per feature (784 for a 28×28 image). No computation.
- ***Hidden layers:*** learn useful intermediate features automatically. "Deep" = many hidden layers.
- ***Output layer:*** size and activation depend on the task (page DL-02).

2 Why Activation Functions?

★★★ MUST KNOW

Without activations, every layer is linear: $W_2(W_1x) = (W_2W_1)x$, which is just ***one*** linear layer. Stacking 100 layers would still only draw straight lines.

Non-linear activations let the network learn curves and complex shapes. A network with one big enough hidden layer can approximate almost any function (universal approximation theorem).

History in 4 lines

1958 Perceptron (single neuron, step function). Can't solve XOR.
1986 Backpropagation popularized → multi-layer networks can be trained.
2012 AlexNet wins ImageNet with a CNN on GPUs → deep learning boom.
2017+ Transformers → ChatGPT, modern vision models.

3 Forward Pass in Matrix Form

★★ IMPORTANT

$$Z_1 = X \cdot W_1 + b_1 \rightarrow A_1 = ReLU(Z_1)$$
$$Z_2 = A_1 \cdot W_2 + b_2 \rightarrow \hat{y} = softmax(Z_2)$$

Tensor	Shape (batch of 64 MNIST images)
X	(64, 784)
W_1, b_1	(784, 128), (128)
A_1	(64, 128)
W_2, b_2	(128, 10), (10)
$\hat{y}$	(64, 10) → one probability per digit

🧠 MEMORIZE

Counting parameters of a Linear layer: ***inputs × outputs + outputs*** (weights + biases).
784 → 128 → 10 network: 784·128 + 128 = 100,480 and 128·10 + 10 = 1,290 → ***101,770 parameters***.

forward pass in NumPy (shapes!)

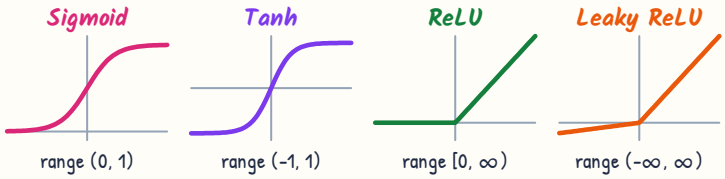
```
import numpy as np
rng = np.random.default_rng(0)
X = rng.random((64, 784))              # 64 images
W1, b1 = rng.normal(0, 0.05, (784, 128)), np.zeros(128)
W2, b2 = rng.normal(0, 0.05, (128, 10)), np.zeros(10)

A1 = np.maximum(0, X @ W1 + b1)      # ReLU
Z2 = A1 @ W2 + b2                     # logits (64, 10)
e = np.exp(Z2 - Z2.max(axis=1, keepdims=True))
probs = e / e.sum(axis=1, keepdims=True)   # softmax
print(probs.shape, probs[0].sum())    # (64, 10) 1.0
```

💡 ***Tip:*** Subtracting the max before `exp` in softmax prevents overflow. It doesn't change the result.

🎤 ***Interview:*** "Why do we need hidden layers?" A single neuron can only draw a straight boundary (it can't solve XOR). Hidden layers + non-linearity combine simple boundaries into complex ones.

1 The Common Activations ★★★ MUST KNOW



Name	Formula	Derivative	Good	Problem
Sigmoid	$1 / (1 + e^{-x})$	$\sigma(x)(1 - \sigma(x))$, max 0.25	output is a probability	saturates → ***vanishing gradients***; not zero-centered
Tanh	$(e^{x} - e^{-x}) / (e^{x} + e^{-x})$	$1 - \tanh^2(x)$, max 1	zero-centered	still saturates for large |x|
ReLU	$\max(0, x)$	1 if $x > 0$, else 0	fast, no saturation for $x > 0$; default	"dying ReLU": neurons stuck at 0
Leaky ReLU	x if $x > 0$, else $0.01x$	1 or 0.01	fixes dying ReLU	one extra choice (slope)
GELU / SiLU	$x \cdot \Phi(x)$ / $x \cdot \sigma(x)$	smooth	smooth ReLU; GELU in transformers, SiLU in YOLO & EfficientNet	a bit slower
Softmax	$e^{z_i} / \sum e^{z_j}$	(used with CE)	multi-class probabilities	output layer only

🧠 MEMORIZE

Which activation where?

Place	Use
Hidden layers	***ReLU*** (start here) · Leaky ReLU / GELU / SiLU
Output: regression	***none*** (linear output)
Output: binary	***sigmoid*** (1 neuron)
Output: multi-class	***softmax*** (C neurons)
Output: multi-label	***sigmoid*** on each of C neurons

Vanishing gradient (why ReLU won)

Backprop multiplies derivatives layer by layer. Sigmoid's derivative is at most ***0.25***, so after 10 layers: $0.25^{10} \approx 0.000001$. Early layers get almost no gradient and stop learning. ReLU's derivative is ***1*** for positive inputs, so gradients pass through unchanged.

⚠️ ***Common mistake:*** In PyTorch, do ***not*** add softmax / sigmoid at the end of your model when using `nn.CrossEntropyLoss` / `nn.BCEWithLogitsLoss`. Those losses apply it internally (more stable). Apply softmax only when you need probabilities for display.

activations in PyTorch

```python
import torch
import torch.nn as nn

x = torch.linspace(-3, 3, 7)    # [-3, -2, -1, 0, 1, 2, 3]
print(torch.relu(x))            # [0, 0, 0, 0, 1, 2, 3]
print(torch.sigmoid(x))
print(nn.LeakyReLU(0.01)(x))
logits = torch.tensor([[2.0, 1.0, 0.1]])
print(torch.softmax(logits, dim=1))  # [[0.659, 0.242, 0.099]]

# as layers inside a model
layer = nn.Sequential(nn.Linear(4, 8), nn.ReLU(),
                      nn.Linear(8, 3))
```

💡 ***Tip:*** Your YOLO models use ***SiLU*** internally. You don't choose it, but now you know what it is.

🎤 ***Interview:*** "What is dying ReLU?" If a neuron's input is always negative, its output and gradient are always 0, so it never updates. Fix: Leaky ReLU, lower learning rate, good initialization.

How big should my network be? ★★

- ***Tabular data:*** 1 to 3 hidden layers, 64 to 256 neurons each (powers of 2 are just convention). But try XGBoost first (ML-18).
- ***Images:*** use a CNN or a pretrained backbone, never a plain MLP on raw pixels (beyond MNIST).
- Start ***small***, make sure it learns (overfit one batch), then grow until validation stops improving.
- Underfitting → bigger/deeper. Overfitting → more data, dropout, weight decay before shrinking.

1 Loss Functions in Deep Learning ★★★ MUST KNOW

Task	Output layer	Loss	PyTorch	Target format
Regression	1 neuron, no activation	MSE (or MAE / Huber)	`nn.MSELoss()`	float numbers
Binary	1 neuron (raw logit)	binary cross-entropy	`nn.BCEWithLogitsLoss()`	float 0. / 1.
Multi-class	C neurons (raw logits)	cross-entropy	`nn.CrossEntropyLoss()`	class index (long): 0..C−1
Multi-label	C neurons (raw logits)	BCE per label	`nn.BCEWithLogitsLoss()`	float 0/1 matrix

💡 **Tip:** For C classes, a random model's cross-entropy loss starts near $\ln(C)$ (10 classes → 2.30). If your first loss is very different, something is wrong.

2 Backpropagation ★★★ MUST KNOW

Backpropagation computes the gradient of the loss with respect to **every weight**, by applying the chain rule from the output back to the input. Then gradient descent updates the weights.

$x, w, b \rightarrow z = wx + b \rightarrow a = f(z) \rightarrow \text{Loss } L$

$$\partial L/\partial w = \partial L/\partial a \cdot \partial a/\partial z \cdot \partial z/\partial w$$

Each layer passes its gradient to the layer before it, multiplying by its own local derivative. PyTorch's **autograd** does all of this for you with `loss.backward()`.

Worked example (1 weight)

$\hat{y} = w \cdot x + b$, $L = (y - \hat{y})^2$
$x = 2$, $y = 10$, $w = 3$, $b = 1$, $lr = 0.01$
Forward: $\hat{y} = 7$, $L = (10 - 7)^2 = 9$
Backward: $\partial L/\partial \hat{y} = -2(y - \hat{y}) = -6$
$\partial L/\partial w = -6 \cdot x = \mathbf{-12}$ $\partial L/\partial b = \mathbf{-6}$
Update: $w = 3 - 0.01 \cdot (-12) = 3.12$
$b = 1 - 0.01 \cdot (-6) = 1.06$
New $\hat{y} = 7.30$ → closer to 10 ✔

🧠 MEMORIZE

One training step, always: ① forward pass → ② compute loss → ③ backward (gradients) → ④ update weights. Repeat for every mini-batch, for many epochs.

3 Neural Network From Scratch (NumPy) ★★ IMPORTANT

2-layer network with manual backprop

```python
import numpy as np
from sklearn.datasets import make_moons

X, y = make_moons(n_samples=500, noise=0.2, random_state=0)
y = y.reshape(-1, 1)                          # (500, 1)
rng = np.random.default_rng(0)
W1 = rng.normal(0, np.sqrt(2 / 2), (2, 16))   # He init (ReLU)
b1 = np.zeros((1, 16))
W2 = rng.normal(0, np.sqrt(1 / 16), (16, 1))
b2 = np.zeros((1, 1))
lr, n = 0.5, len(X)

for epoch in range(3000):
    # ---- forward ----
    Z1 = X @ W1 + b1
    A1 = np.maximum(0, Z1)                    # ReLU
    Z2 = A1 @ W2 + b2
    p = 1 / (1 + np.exp(-Z2))                 # sigmoid
    loss = -np.mean(y * np.log(p + 1e-9)
                    + (1 - y) * np.log(1 - p + 1e-9))
    # ---- backward (chain rule) ----
    dZ2 = (p - y) / n                         # BCE + sigmoid
    dW2 = A1.T @ dZ2
    db2 = dZ2.sum(axis=0, keepdims=True)
    dA1 = dZ2 @ W2.T                          # send error back
    dZ1 = dA1 * (Z1 > 0)                      # ReLU derivative
    dW1 = X.T @ dZ1
    db1 = dZ1.sum(axis=0, keepdims=True)
    # ---- update ----
    W1 -= lr * dW1; b1 -= lr * db1
    W2 -= lr * dW2; b2 -= lr * db2

print("loss:", round(loss, 3), "acc:", ((p > 0.5) == y).mean())
```

What to notice

- The backward pass goes in **reverse order** of the forward pass.
- `dZ2 = p - y`: the same simple error term as logistic regression.
- `dA1 = dZ2 @ W2.T`: the error is sent back through the weights.
- `(Z1 > 0)` is ReLU's derivative: gradient passes only where the neuron was active.
- A straight line can't separate the moons; this network can, thanks to the hidden ReLU layer.

💡 **Tip:** Build this once to understand backprop. After that, let PyTorch do it.

1 Optimizers ★★ IMPORTANT

Optimizer	Idea	Typical settings
SGD	$w = w - lr \cdot g$ (plain gradient step on a mini-batch)	lr 0.01 to 0.1
SGD + Momentum	keeps a "velocity": $v = \beta v + g$, $w = w - lr \cdot v$. Rolls through small bumps, smooths zig-zags.	momentum 0.9; strong for CNNs trained from scratch
RMSprop	divides each step by a running average of squared gradients → each weight gets its own step size	lr 1e-3
Adam ✅	momentum + RMSprop + bias correction. Works well with little tuning.	lr = 1e-3, β_1 0.9, β_2 0.999
AdamW ✅	Adam with ***decoupled weight decay*** (correct L2 regularization). Modern default.	lr 1e-3, weight_decay 0.01

🧠 MEMORIZE

Adam update: $m = \beta_1 m + (1-\beta_1) g$ $v = \beta_2 v + (1-\beta_2) g^2$ $\hat{m} = m/(1-\beta_1^t)$, $\hat{v} = v/(1-\beta_2^t)$ $\boldsymbol{w = w - lr \cdot \hat{m} / (\sqrt{\hat{v}} + \varepsilon)}$

Rule of thumb: start with **Adam/AdamW, lr = 1e-3**. Fine-tuning a pretrained model: smaller lr (1e-4 to 1e-5).

2 Learning Rate & Schedules ★★★ MUST KNOW

The learning rate is the most important hyperparameter. Too high → loss jumps or becomes NaN. Too low → painfully slow, stuck.

Schedulers change the lr during training: large steps early, small steps later to settle into a good minimum.

- **StepLR:** multiply lr by 0.1 every N epochs.
- **CosineAnnealingLR:** smooth decrease following a cosine curve (popular, YOLO uses cosine/linear decay).
- **ReduceLROnPlateau:** lower lr when val loss stops improving.
- **Warmup:** start tiny for the first few epochs, then increase (stabilizes early training).

optimizer + scheduler in PyTorch

```python
import torch
import torch.nn as nn
model = nn.Sequential(nn.Linear(10, 32), nn.ReLU(),
                      nn.Linear(32, 2))
opt = torch.optim.AdamW(model.parameters(), lr=1e-3,
                        weight_decay=0.01)
# alternatives:
# opt = torch.optim.SGD(model.parameters(), lr=0.01,
#                       momentum=0.9)
# opt = torch.optim.Adam(model.parameters(), lr=1e-3)

sched = torch.optim.lr_scheduler.CosineAnnealingLR(
    opt, T_max=20)              # 20 = number of epochs
for epoch in range(20):
    # ... train one epoch (page DL-07) ...
    sched.step()                # once per epoch
    print(epoch, sched.get_last_lr())
```

3 Batch Size & Epochs ★★ IMPORTANT

🧠 MEMORIZE

Epoch = one full pass over the training set.
Batch size = samples per weight update (16, 32, 64, 128).
Iterations per epoch = ⌈ N / batch size ⌉ → 60,000 images, batch 64 → 938 iterations.

- Bigger batch → faster per epoch, more GPU memory, smoother gradients.
- "CUDA out of memory" → **reduce batch size** (or image size).
- Train until validation loss stops improving (early stopping), not a fixed big number.

4 Weight Initialization ★ GOOD TO KNOW

- **Never all zeros:** every neuron would learn the same thing (symmetry problem).
- **Xavier / Glorot:** for sigmoid / tanh. Variance ≈ 2 / (fan_in + fan_out).
- **He / Kaiming:** for ReLU. Variance = 2 / fan_in.
- Goal: keep the signal from shrinking or exploding as it passes through layers.
- PyTorch layers already use a sensible default (Kaiming-uniform style), so you rarely set this by hand.

Tuning priority (what to try first)

① learning rate → ② batch size / epochs → ③ model size (layers, width) → ④ regularization (dropout, weight decay, augmentation) → ⑤ optimizer type. Change **one thing at a time** and write down the result.

1 Read Your Learning Curves ★★★ MUST KNOW



Good: both losses fall and stay close.
Overfitting: train keeps falling, val turns ***up***. Stop at the lowest val loss.
Underfitting: both stay high → bigger model, train longer, higher lr.
Loss jumping / NaN: lr too high, bad data (NaN values), or missing normalization.

2 Fighting Overfitting in Deep Learning ★★★ MUST KNOW

More / augmented data
The best fix. For images: flips, rotations, crops, color changes (page CV-03). Creates new training examples for free.

Dropout
During training, randomly turn off a fraction ***p*** of neurons (0.2 to 0.5) each step. The network can't rely on any single neuron. Off during evaluation.

Weight decay (L2)
Penalizes large weights, like Ridge. Set in the optimizer: `weight_decay=1e-4` to `1e-2`.

Early stopping
Track val loss each epoch; keep the best model; stop if no improvement for ***patience*** epochs (e.g. 5 to 10).

Smaller model
Fewer layers / neurons. Or use transfer learning so fewer weights need training.

Batch Normalization
Normalizes each layer's outputs over the mini-batch (mean 0, std 1), then learns scale γ and shift β. Trains faster, more stable, slight regularization.

3 Vanishing & Exploding Gradients ★★ IMPORTANT

Vanishing: gradients shrink layer by layer → early layers stop learning (sigmoid/tanh in deep nets).
Exploding: gradients grow huge → weights become NaN (deep nets, RNNs, high lr).

🧠 MEMORIZE

Fixes: ReLU-family activations · He initialization · ***Batch Normalization*** · ***residual (skip) connections*** (ResNet, page CV-04) · ***gradient clipping***:
`torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)`

⚠️ ***Common mistake:*** Forgetting `model.train()` / `model.eval()`. Dropout and BatchNorm behave differently in training and evaluation. Always call `model.eval()` before validating or predicting.

4 Debugging Checklist ★★★ MUST KNOW

1. ***Look at your data:*** plot a few images with their labels. Are labels correct?
2. ***Check shapes*** at every step: `print(x.shape)`.
3. ***Check the first loss*** ≈ ln(C) for C classes.
4. ***Overfit one small batch*** (e.g. 32 samples) to ~100% accuracy. If you can't, there's a bug in the model or loop.
5. Then train on all data; watch train ***and*** val curves.
6. NaN loss → lower lr, check for NaN inputs, check normalization.
7. Fix random seeds to make runs comparable: `torch.manual_seed(42)`.
8. Change ***one thing at a time***, keep notes of every experiment.

🎤 ***Interview:*** "Explain dropout." It randomly zeroes a fraction of activations during training (PyTorch scales the rest by 1/(1−p) so the expected value stays the same). It acts like training many smaller networks and averaging them. At test time all neurons are used.

1 Tensors ★★★ MUST KNOW

A ***tensor*** is PyTorch's NumPy array, but it can run on a ***GPU*** and track gradients.

tensor basics

```python
import torch
import numpy as np

a = torch.tensor([[1., 2.], [3., 4.]])
b = torch.zeros(2, 3); c = torch.rand(2, 3)
print(a.shape, a.dtype)        # torch.Size([2, 2]) float32
print(a @ a, a * a)            # matrix mult, element-wise
print(a.sum(), a.mean(dim=0))  # reductions

x = torch.rand(8, 3, 32, 32)   # (N, C, H, W) images
print(x.view(8, -1).shape)     # flatten → (8, 3072)
print(x.permute(0, 2, 3, 1).shape)  # → (N, H, W, C)
print(x[0].unsqueeze(0).shape) # add batch dim → (1,3,32,32)

t = torch.from_numpy(np.ones(3))   # NumPy → tensor
back = t.numpy()                   # tensor → NumPy

# pick the fastest device available
device = ("cuda" if torch.cuda.is_available()
          else "mps" if torch.backends.mps.is_available()
          else "cpu")              # mps = Apple Silicon Mac
x = x.to(device)
```

🧠 MEMORIZE

Image tensors in PyTorch are (N, C, H, W): batch, channels, height, width. NumPy / OpenCV / matplotlib use (H, W, C).

2 Autograd ★★ IMPORTANT

automatic gradients

```python
x = torch.tensor(2.0, requires_grad=True)
y = x ** 2 + 3 * x       # y = x² + 3x
y.backward()             # compute dy/dx
print(x.grad)            # 2x + 3 = tensor(7.)
```

PyTorch records every operation in a graph, then `backward()` applies the chain rule automatically. Model weights have `requires_grad=True` by default.

⚠️ ***Common mistake:*** Gradients ***accumulate*** (add up) across `backward()` calls. That's why every training step starts with `optimizer.zero_grad()`.

3 Building a Model ★★★ MUST KNOW

nn.Module (the standard way)

```python
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, in_dim=784, hidden=128, n_classes=10):
        super().__init__()                  # always call this
        self.net = nn.Sequential(
            nn.Flatten(),                   # (N,1,28,28)→(N,784)
            nn.Linear(in_dim, hidden), nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(hidden, n_classes),  # raw logits
        )

    def forward(self, x):                   # defines the pass
        return self.net(x)

model = MLP()
out = model(torch.rand(4, 1, 28, 28))      # calls forward
print(out.shape)                           # (4, 10)
print(sum(p.numel() for p in model.parameters()))  # 101770
```

4 Dataset & DataLoader ★★★ MUST KNOW

custom Dataset for your own data

```python
from torch.utils.data import Dataset, DataLoader

class TabularDataset(Dataset):
    def __init__(self, X, y):
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.long)
    def __len__(self):                  # number of samples
        return len(self.X)
    def __getitem__(self, i):           # return ONE sample
        return self.X[i], self.y[i]

ds = TabularDataset(np.random.rand(100, 5),
                    np.random.randint(0, 2, 100))
loader = DataLoader(ds, batch_size=16, shuffle=True)
xb, yb = next(iter(loader))
print(xb.shape, yb.shape)   # (16, 5) (16,)
```

Dataset knows how to get ***one*** sample (`__len__`, `__getitem__`).

DataLoader groups samples into ***batches***, shuffles them, and can load in parallel (`num_workers`).

🧠 MEMORIZE

shuffle=True for training, ***shuffle=False*** for validation/test. Classification labels → `torch.long`; features → `torch.float32`.

💡 ***Tip:*** For images in folders, you don't need a custom class: `torchvision.datasets.ImageFolder` does it (page CV-05).

1 The 5 Lines Inside Every Training Loop 🧠 MEMORIZE

🧠 MEMORIZE

the heart of all deep learning

```
optimizer.zero_grad()             # 1. clear old gradients
outputs = model(images)           # 2. forward pass
loss = criterion(outputs, labels)  # 3. compute loss
loss.backward()                   # 4. backprop: gradients
optimizer.step()                  # 5. update weights
```

Same steps as gradient descent (predict → loss → gradients → update), plus clearing old gradients first. ***Evaluation*** = forward pass only, inside `model.eval()` + `torch.no_grad()`.

⚠️ ***Common mistake:*** softmax before `CrossEntropyLoss`, forgetting `zero_grad()` or `model.eval()`.

2 Complete Example: Digit Classifier ★★★ MUST KNOW

mnist_train.py (runs on CPU in a few minutes, faster on Colab GPU)

```
import torch, torch.nn as nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

torch.manual_seed(42)
device = "cuda" if torch.cuda.is_available() else "cpu"

# 1. Data: ToTensor scales pixels to [0, 1]; Normalize uses MNIST mean/std
tfm = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.1307,), (0.3081,))])
train_ds = datasets.MNIST("data", train=True, download=True, transform=tfm)
test_ds = datasets.MNIST("data", train=False, download=True, transform=tfm)
train_dl = DataLoader(train_ds, batch_size=64, shuffle=True)
test_dl = DataLoader(test_ds, batch_size=256, shuffle=False)

# 2. Model, loss, optimizer
model = nn.Sequential(nn.Flatten(), nn.Linear(784, 128), nn.ReLU(), nn.Dropout(0.2),
                      nn.Linear(128, 10)).to(device)
criterion = nn.CrossEntropyLoss()                 # expects raw logits + class indices
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

def train_one_epoch():
    model.train()                                 # dropout ON
    total_loss = 0
    for images, labels in train_dl:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        loss = criterion(model(images), labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * len(images)    # .item() → Python number
    return total_loss / len(train_dl.dataset)

@torch.no_grad()                                  # no gradients during evaluation
def evaluate(loader):
    model.eval()                                  # dropout OFF
    correct, total_loss = 0, 0
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        total_loss += criterion(outputs, labels).item() * len(images)
        correct += (outputs.argmax(dim=1) == labels).sum().item()   # argmax = predicted class
    n = len(loader.dataset)
    return total_loss / n, correct / n

# 3. Train, keep the best model (real projects: use a separate validation set)
best_acc = 0
for epoch in range(1, 6):
    train_loss = train_one_epoch()
    val_loss, val_acc = evaluate(test_dl)
    print(f"epoch {epoch}: train {train_loss:.3f} | val {val_loss:.3f} | acc {val_acc:.3f}")
    if val_acc > best_acc:
        best_acc = val_acc
        torch.save(model.state_dict(), "best_model.pt")   # save weights only

# 4. Load and predict on one image
model.load_state_dict(torch.load("best_model.pt", map_location=device))
model.eval()
image, label = test_ds[0]
with torch.no_grad():
    probs = torch.softmax(model(image.unsqueeze(0).to(device)), dim=1)
print("true:", label, "predicted:", probs.argmax().item(), "confidence:", round(probs.max().item(), 3))
```

💡 ***Tip:*** `.to(device)` : model ***and*** data must be on the same device. `state_dict()` = all weights (standard way to save). Expect ~97 to 98% accuracy after 5 epochs; a CNN (page CV-03) reaches ~99%.

1 What a Computer Sees ★★★ MUST KNOW

- An image is a ***grid of numbers*** (pixels). Each pixel is 0 (dark) to 255 (bright).
- ***Grayscale:*** 1 channel → shape (H, W). ***Color (RGB):*** 3 channels → (H, W, 3).
- A 224×224 RGB image = 224 × 224 × 3 = ***150,528 numbers***.
- Before training: convert to float, scale to ***[0, 1]*** (÷255), then often ***normalize*** per channel: (x − mean) / std.
- PyTorch wants (C, H, W); `transforms.ToTensor()` converts (H, W, C) uint8 → (C, H, W) float in [0, 1].

A 4×4 grayscale patch

12	40	150	230
25	90	180	250
35	110	200	255
60	120	215	255

dark on the left, bright on the right → a ***vertical edge***

2 Why Not a Normal Neural Network (MLP)? ★★★ MUST KNOW

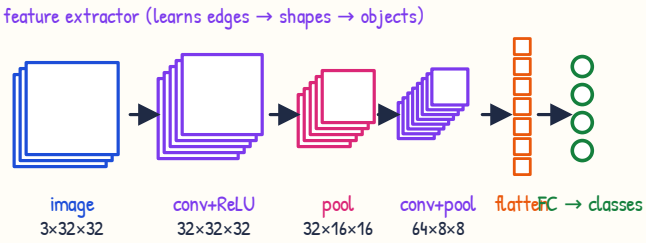
Problems with MLPs on images

- ***Too many weights:*** 150,528 inputs × 1,000 hidden neurons = ***150 million*** weights in the first layer alone → slow, overfits.
- ***Loses spatial structure:*** flattening destroys which pixels are neighbors.
- ***Not shift-invariant:*** a cat in the top-left and bottom-right would need to be learned separately.

How CNNs fix it

- ***Local connections:*** each neuron looks at a small patch (e.g. 3×3), not the whole image.
- ***Weight sharing:*** the same small filter slides over the whole image → few parameters (a 3×3 filter on RGB has only 28).
- ***Translation invariance:*** a feature detector works anywhere in the image.
- ***Hierarchy:*** early layers find edges → middle layers find textures and parts → deep layers find whole objects.

3 The Big Picture of a CNN



load and inspect an image

```python
from PIL import Image
import numpy as np
from torchvision import transforms

# real photo: img = Image.open("photo.jpg").convert("RGB")
img = Image.new("RGB", (64, 48), color=(200, 30, 30))
arr = np.array(img)
print(arr.shape, arr.dtype)   # (48, 64, 3) uint8, values 0-255
t = transforms.ToTensor()(img)
print(t.shape)                # (3, 48, 64), values now 0-1
```

OpenCV (camera / Raspberry Pi) uses BGR, not RGB

```python
import cv2
frame = cv2.imread("bottle.jpg")   # (H, W, 3) in BGR!
rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)  # → RGB
small = cv2.resize(rgb, (224, 224))  # size is (W, H)
cap = cv2.VideoCapture(0)            # webcam
ok, frame = cap.read()               # one frame
```

🎤 ***Interview:*** "Why do CNNs work better than MLPs on images?" Local connectivity and weight sharing use far fewer parameters and keep spatial structure, and the same filter detects a pattern anywhere.

💡 ***Tip:*** Always plot a few images after loading (`plt.imshow(arr)`). Wrong color order (BGR vs RGB in OpenCV) and wrong shapes are very common bugs.

⚠️ ***Common mistake:*** Feeding BGR images to a model trained on RGB. Colors swap (red ↔ blue) and accuracy drops. Ultralytics handles this for you; your own PyTorch code doesn't.

1 Convolution ★★★ MUST KNOW

A **filter** (kernel), e.g. 3×3, **slides** over the image. At each position, multiply the overlapping numbers element-wise and **add them up** → one output number. The output grid is called a feature map.

10	10	0
10	10	0
10	10	0

⊙

1	0	-1
1	0	-1
1	0	-1

= **30**

patch · vertical-edge filter: (10+10+10)·1 + 0 + (0+0+0)·(−1) = **30** → strong response = "edge here!"

- In a CNN, filter values are **learned** by backprop, not designed by hand.
- A filter spans **all input channels**: on an RGB image a 3×3 filter is really 3×3×3.
- **Number of filters** = number of output channels (feature maps).
- After the convolution: add bias, then **ReLU**.

2 Padding, Stride and Output Size ★★★ MUST KNOW

Padding (P): add zeros around the border so the output doesn't shrink and edge pixels are used. "**same**" padding keeps size (P = 1 for a 3×3 filter, stride 1). "**valid**" = no padding.

Stride (S): how many pixels the filter jumps each move. Stride 2 → output about half the size.

Kernel size (K): usually 3×3 (sometimes 1×1, 5×5, 7×7).

🧠 MEMORIZE

$$\text{Output} = \lfloor (W - K + 2P) / S \rfloor + 1$$

$$\text{Conv params} = (K \cdot K \cdot C_{in} + 1) \cdot C_{out}$$

Input W	K	P	S	Output
32	3	1	1	**32** (same)
32	3	0	1	**30**
32	3	1	2	**16**
224	7	3	2	**112**

Conv2d(3, 32, 3): (3·3·3 + 1)·32 = **896** params

3 Pooling ★★★ MUST KNOW

1	**6**	2	0
3	4	**7**	1
0	2	**9**	5
5	1	3	8

→ max pool 2×2 →

6	7
5	9

- **Max pooling** keeps the strongest response in each window. 2×2, stride 2 → halves width and height.
- Reduces computation, adds a little shift tolerance. **No learnable parameters.**
- **Average pooling** takes the mean. **Global Average Pooling** averages each whole feature map to one number → replaces big flatten + dense layers (used in ResNet).

4 Other Key Ideas ★★ IMPORTANT

- **Receptive field:** the region of the original image a neuron "sees". Stacking layers grows it: two 3×3 convs see 5×5, three see 7×7.
- **1×1 convolution:** mixes channels without looking at neighbors. Used to change the number of channels cheaply.
- **Feature maps get smaller but deeper:** 3×224×224 → … → 512×7×7. Spatial size ↓, channels ↑.

check output shapes yourself

```python
import torch, torch.nn as nn
x = torch.rand(1, 3, 32, 32)          # one RGB 32×32 image
conv = nn.Conv2d(3, 32, kernel_size=3, padding=1)
print(conv(x).shape)                   # (1, 32, 32, 32)
print(nn.MaxPool2d(2)(conv(x)).shape)  # (1, 32, 16, 16)
print(sum(p.numel() for p in conv.parameters()))  # 896
down = nn.Conv2d(3, 8, 3, stride=2, padding=1)
print(down(x).shape)                   # (1, 8, 16, 16)
```

🎤 **Interview:** "Calculate the output size of a 64×64 input, 5×5 kernel, padding 2, stride 1." (64 − 5 + 4)/1 + 1 = 64.

Practice: track the shapes (answers on the right) ★★

Input **3×64×64** → Conv(3→16, K=3, P=1, S=1) → MaxPool 2 → Conv(16→32, K=3, P=0, S=1) → MaxPool 2 → Flatten → Linear(?, 6). What is "?" and how many parameters does the first conv have?

16×64×64 → 16×32×32 → 32×30×30 → 32×15×15
Flatten: 32·15·15 = **7,200**
Conv1 params: (3·3·3 + 1)·16 = **448**

1 The Standard CNN Pattern ★★★ MUST KNOW

Image → [Conv → BatchNorm → ReLU → MaxPool] × N → Flatten / Global Avg Pool → Linear (+ Dropout) → logits

a CNN for CIFAR-10 (10 classes, 32×32 RGB)

```
import torch.nn as nn

class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        # one block = conv → BN → ReLU → pool
        def block(c_in, c_out):
            return nn.Sequential(
                nn.Conv2d(c_in, c_out, 3, padding=1),
                nn.BatchNorm2d(c_out),
                nn.ReLU(),
                nn.MaxPool2d(2))        # halves H and W
        self.features = nn.Sequential(
            block(3, 32),       # 3x32x32  → 32x16x16
            block(32, 64),      # 32x16x16 → 64x8x8
            block(64, 128))     # 64x8x8   → 128x4x4
        self.classifier = nn.Sequential(
            nn.Flatten(),       # 128*4*4 = 2048
            nn.Dropout(0.3),
            nn.Linear(128 * 4 * 4, 256), nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(256, num_classes))  # logits

    def forward(self, x):
        return self.classifier(self.features(x))
```

Layer	Output shape
input	3 × 32 × 32
block 1	32 × 16 × 16
block 2	64 × 8 × 8
block 3	128 × 4 × 4
flatten	2048
linear	256 → 10

🧠 MEMORIZE

Channels ***double*** (32 → 64 → 128) while size ***halves***. Track shapes with a table like this; most CNN bugs are shape mismatches in the first Linear layer.

💡 ***Tip:*** Use ***exactly the same training loop*** as page DL-07. Only the model and the data change.

2 Data Augmentation ★★★ MUST KNOW

Create new training images by randomly changing existing ones: flips, crops, rotations, brightness. The model sees "new" data every epoch → less overfitting, better generalization.

- Apply to the ***training set only***. Val/test get only resize + normalize.
- Augmentations must keep the label true. Flipping a "6" makes it a "9"!
- For ***detection***, boxes must move with the image (YOLO does this for you, plus ***mosaic***: 4 images stitched into one).

train vs test transforms + training

```
import torch
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# CIFAR-10 channel mean and std
mean = (0.4914, 0.4822, 0.4465)
std = (0.2470, 0.2435, 0.2616)
train_tfm = transforms.Compose([
    transforms.RandomCrop(32, padding=4),   # random shift
    transforms.RandomHorizontalFlip(),      # mirror, p=0.5
    transforms.ColorJitter(0.2, 0.2, 0.2),  # light changes
    transforms.ToTensor(),
    transforms.Normalize(mean, std)])
test_tfm = transforms.Compose([transforms.ToTensor(),
                               transforms.Normalize(mean, std)])

train_ds = datasets.CIFAR10("data", train=True,
                            download=True, transform=train_tfm)
test_ds = datasets.CIFAR10("data", train=False,
                           download=True, transform=test_tfm)
train_dl = DataLoader(train_ds, batch_size=128, shuffle=True)
test_dl = DataLoader(test_ds, batch_size=256)

device = "cuda" if torch.cuda.is_available() else "cpu"
model = SimpleCNN().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3,
                              weight_decay=1e-4)
# run train_one_epoch() / evaluate() from DL-07, 20-30 epochs
```

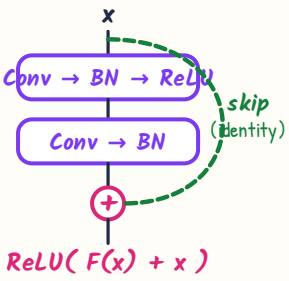
🛠️ ***For your project:*** For waste images: horizontal flips, small rotations, brightness/contrast changes (different lighting) and random crops are all safe. Collect photos in different places and lighting too; augmentation can't replace real variety.

🎤 ***Interview:*** "How does BatchNorm behave at test time?" It uses running averages of mean and variance collected during training instead of the current batch, which is why `model.eval()` matters.

1 Timeline of Milestones ★★ IMPORTANT

Year	Model	Key idea (remember this)
1998	**LeNet-5**	first successful CNN, read handwritten digits (conv → pool → conv → pool → dense)
2012	**AlexNet**	won ImageNet by a huge margin; **ReLU**, **dropout**, data augmentation, trained on **GPUs** → started the deep learning boom
2014	**VGG-16/19**	simple and deep: only stacked **3×3 convs**. Two 3×3 = one 5×5 view with fewer params. Very heavy (~138M params)
2014	**GoogLeNet / Inception**	parallel 1×1, 3×3, 5×5 filters in one block; 1×1 convs to save compute
2015	**ResNet** ⭐	skip (residual) connections → trained 152 layers; fixed the degradation problem of very deep nets
2017	**MobileNet**	**depthwise separable convolutions** → small and fast, built for phones / edge devices
2019	**EfficientNet**	scales depth, width and image resolution together (compound scaling)
2020	**Vision Transformer (ViT)**	splits the image into patches and uses attention instead of convolutions; needs lots of data ★

2 ResNet and Skip Connections ★★★ MUST KNOW



The problem: adding more layers to a plain CNN eventually made **training** error worse (not just test error). Very deep plain networks are hard to optimize; gradients vanish.

The fix: add the input **x** directly to the block's output: **y = F(x) + x**. The block only has to learn the **difference** (residual) F(x). If a layer isn't useful, it can learn F(x) ≈ 0 and pass x through unchanged.

Why it works: gradients flow straight back through the "+ x" shortcut, so even the earliest layers get a strong signal.

a residual block

```python
import torch.nn as nn
class ResidualBlock(nn.Module):
    def __init__(self, ch):
        super().__init__()
        self.f = nn.Sequential(
            nn.Conv2d(ch, ch, 3, padding=1, bias=False), nn.BatchNorm2d(ch), nn.ReLU(),
            nn.Conv2d(ch, ch, 3, padding=1, bias=False), nn.BatchNorm2d(ch))
    def forward(self, x):
        return nn.functional.relu(self.f(x) + x)   # skip connection
```

3 Depthwise Separable Conv ★★ IMPORTANT

A normal 3×3 conv mixes space and channels at once. MobileNet splits it in two cheap steps:

1. **Depthwise:** one 3×3 filter **per channel** (no channel mixing).
2. **Pointwise:** a 1×1 conv to mix channels.

About **8 to 9× less computation** for 3×3 kernels, with a small accuracy cost. Great for a **Raspberry Pi**.

depthwise separable in PyTorch

```python
def dw_separable(c_in, c_out):
    return nn.Sequential(
        # depthwise: groups=c_in → one filter per channel
        nn.Conv2d(c_in, c_in, 3, padding=1, groups=c_in),
        # pointwise: 1×1 conv mixes channels
        nn.Conv2d(c_in, c_out, 1))
```

4 Which Backbone for You?

Situation	Pick
Learning / first transfer learning	**ResNet-18 / 50**
Phone, Raspberry Pi, real-time	**MobileNetV3**, EfficientNet-B0
Best accuracy, have a GPU	EfficientNet, ConvNeXt, ViT
Object detection	**YOLO** (its own CNN backbone)

🎤 **Interview:** "What problem does ResNet solve?" The degradation problem: very deep plain networks train worse. Skip connections let gradients flow and let blocks learn small residual changes.

💡 **Tip:** You almost never design architectures yourself. You pick a proven one and fine-tune it (next page).

1 The Idea ★★★ MUST KNOW

Take a model already trained on a huge dataset (***ImageNet***: ~1.2 million images, 1,000 classes) and reuse what it learned for ***your*** task. Its early layers already detect edges, textures and shapes, which are useful for almost any image problem.

- Works with ***small datasets*** (a few hundred images per class).
- Trains ***much faster*** and reaches ***higher accuracy*** than training from scratch.
- This is how most real computer vision projects are built. YOLO training also starts from pretrained weights.

Two strategies

① ***Feature extraction:*** freeze all pretrained layers, replace and train only the last layer (the "head"). Fast; good for tiny datasets.

② ***Fine-tuning:*** after ①, unfreeze some or all layers and continue training with a ***small learning rate*** (1e-4 to 1e-5) so pretrained knowledge isn't destroyed. Best accuracy.

Your data	Similar to ImageNet (photos of objects)	Very different (X-rays, satellite, spectrograms)
Small	feature extraction	feature extraction from earlier layers, then careful fine-tuning
Large	fine-tune the whole model	fine-tune everything (or train from scratch)

2 Code: Fine-tune ResNet-18 on Your Own Folders ★★★ MUST KNOW

Folder structure

```
waste/
├── train/
│   ├── plastic/ *.jpg
│   ├── paper/
│   ├── glass/ ...
├── val/
│   ├── plastic/
│   └── ...
└── test/ ...
```

ImageFolder uses folder names as class labels automatically (sorted alphabetically).

MEMORIZE

ImageNet normalization:
mean = [0.485, 0.456, 0.406]
std = [0.229, 0.224, 0.225]
Pretrained models expect it.

transfer_learning.py

```python
import torch, torch.nn as nn
from torchvision import datasets, transforms, models
from torch.utils.data import DataLoader

mean, std = [0.485, 0.456, 0.406], [0.229, 0.224, 0.225]
train_tfm = transforms.Compose([
    transforms.RandomResizedCrop(224, scale=(0.7, 1.0)),
    transforms.RandomHorizontalFlip(),
    transforms.ColorJitter(0.3, 0.3, 0.3),
    transforms.ToTensor(), transforms.Normalize(mean, std)])
eval_tfm = transforms.Compose([
    transforms.Resize(256), transforms.CenterCrop(224),
    transforms.ToTensor(), transforms.Normalize(mean, std)])

train_ds = datasets.ImageFolder("waste/train", transform=train_tfm)
val_ds = datasets.ImageFolder("waste/val", transform=eval_tfm)
train_dl = DataLoader(train_ds, batch_size=32, shuffle=True, num_workers=2)
val_dl = DataLoader(val_ds, batch_size=64)
print(train_ds.classes)                    # ['glass', 'paper', 'plastic', ...]

device = "cuda" if torch.cuda.is_available() else "cpu"
model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)  # pretrained

# Phase 1: freeze everything, replace the head
for p in model.parameters():
    p.requires_grad = False
model.fc = nn.Linear(model.fc.in_features, len(train_ds.classes))  # new, trainable
model = model.to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.AdamW(model.fc.parameters(), lr=1e-3)
# ... train 5-10 epochs with the DL-07 loop ...

# Phase 2: unfreeze all, fine-tune gently
for p in model.parameters():
    p.requires_grad = True
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4, weight_decay=1e-4)
# ... train 10-20 more epochs, keep the best on val accuracy ...
```

Predict one new image

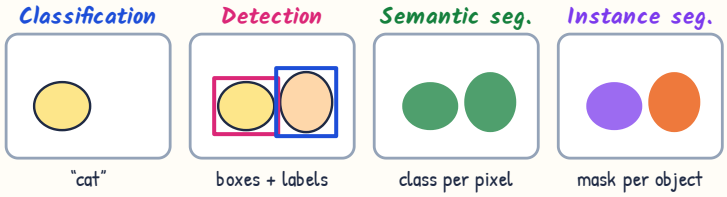
inference

```python
from PIL import Image
model.eval()
img = eval_tfm(Image.open("bottle.jpg").convert("RGB"))
with torch.no_grad():
    logits = model(img.unsqueeze(0).to(device))   # add batch dim
    probs = torch.softmax(logits, dim=1)[0]
top = probs.argmax().item()
print(train_ds.classes[top], round(probs[top].item(), 3))
```

⚠️ ***Common mistake:*** Using a different preprocessing at test time than in training (e.g. forgetting Normalize) → accuracy collapses.

💡 ***Tip:*** Report a ***confusion matrix*** and per-class F1 on the test set (page ML-10). It shows which classes get confused (e.g. plastic vs glass).

1 Computer Vision Tasks ★★★ MUST KNOW



Classification: one label for the whole image. "What is in it?"

Detection: a ***box + label + confidence*** for every object. "What and where?"

Semantic segmentation: a class for every pixel; two bottles = one "bottle" region.

Instance segmentation: a separate pixel mask for each object (YOLO-seg).

2 Bounding Box Formats ★★★ MUST KNOW

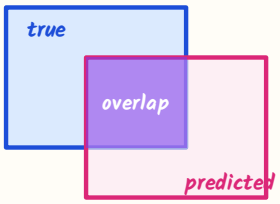
Format	Values	Used by
xyxy	x1, y1 (top-left), x2, y2 (bottom-right), pixels	torchvision, NMS, drawing
xywh (COCO)	x1, y1 (top-left), width, height, pixels	COCO JSON
YOLO ⭐	x_center, y_center, width, height, all divided by image size (0 to 1)	YOLO label .txt files

Image origin (0, 0) is the ***top-left*** corner; y grows ***downward***.

🧠 MEMORIZE

Pixel xyxy → YOLO (image W×H):
cx = (x1 + x2) / 2 / W cy = (y1 + y2) / 2 / H
w = (x2 − x1) / W h = (y2 − y1) / H
Example 640×480 image, box (100, 120, 300, 360):
cx = 200/640 = 0.3125, cy = 240/480 = 0.5,
w = 200/640 = 0.3125, h = 240/480 = 0.5
label line: `0 0.3125 0.5 0.3125 0.5`

3 IoU ★★★ MUST KNOW



$$IoU = \frac{\text{overlap area}}{\text{union area}}$$

1 = perfect, 0 = no overlap

Intersection over Union measures how well two boxes overlap. Used to decide if a prediction is correct (IoU ≥ 0.5 is a common rule), in NMS, and in box losses.

IoU from scratch

```python
def iou(a, b):                    # boxes as [x1, y1, x2, y2]
    x1, y1 = max(a[0], b[0]), max(a[1], b[1])
    x2, y2 = min(a[2], b[2]), min(a[3], b[3])
    inter = max(0, x2 - x1) * max(0, y2 - y1)
    area_a = (a[2] - a[0]) * (a[3] - a[1])
    area_b = (b[2] - b[0]) * (b[3] - b[1])
    return inter / (area_a + area_b - inter)

print(iou([0, 0, 10, 10], [5, 5, 15, 15]))  # 25/175 = 0.143
```

One-stage vs Two-stage detectors ★★

	Two-stage	One-stage
How	① propose regions, ② classify each	predict boxes + classes in a ***single pass***
Examples	R-CNN, Fast/Faster R-CNN, Mask R-CNN	***YOLO***, SSD, RetinaNet
Trade-off	accurate, slower	fast, real-time, now also very accurate

4 Non-Max Suppression (NMS) ★★★ MUST KNOW

Detectors often predict ***several overlapping boxes*** for the same object. NMS keeps the best one:

1. Drop boxes below the ***confidence threshold*** (e.g. 0.25).
2. Sort the rest by confidence (per class).
3. Keep the top box; ***remove*** others with IoU > threshold (e.g. 0.5 to 0.7) with it.
4. Repeat with the next remaining box.

NMS with torchvision

```python
import torch
from torchvision.ops import nms
boxes = torch.tensor([[100, 100, 200, 200],
                      [105, 102, 205, 198],   # duplicate
                      [300, 300, 380, 400]], dtype=torch.float)
scores = torch.tensor([0.90, 0.75, 0.80])
print(nms(boxes, scores, iou_threshold=0.5))  # tensor([0, 2])
```

YOLO26 is designed to be ***NMS-free*** (end-to-end): the model learns to output one box per object, which makes deployment simpler and faster. Older YOLOs (v5, v8, YOLO11) use NMS.

Anchors (★): predefined box shapes the model adjusts. YOLOv2 to v5 used anchors; ***YOLOv8 and later are anchor-free*** (they predict object centers and box sizes directly).

🎤 ***Interview:*** "What is IoU and how is it used?" Overlap area ÷ union area of two boxes. It decides TP vs FP in evaluation, removes duplicates in NMS, and appears in box regression losses.

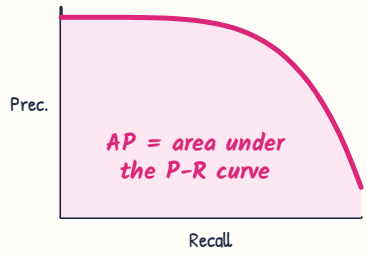
1 TP, FP, FN in Detection ★★★ MUST KNOW

Outcome	Meaning
True Positive	predicted box has **IoU ≥ threshold** (e.g. 0.5) with a real object of the **same class**, and that object wasn't already matched
False Positive	wrong class, IoU too low, a **duplicate** box, or a box on background
False Negative	a real object that **no** prediction matched (missed)
True Negative	not used: there are endless "empty" boxes

🧠 MEMORIZE

Precision = TP / (TP + FP) → "how many of my boxes are right?"
Recall = TP / (TP + FN) → "how many real objects did I find?"
Confidence threshold ↑ → precision ↑, recall ↓

2 AP and mAP ★★★ MUST KNOW



1. For one class, sort all predictions by confidence.
2. Go down the list; after each prediction compute precision and recall → this traces the **Precision-Recall curve**.
3. **AP (Average Precision)** = area under that curve (0 to 1). High AP = high precision at every recall level.
4. **mAP** = mean of AP over **all classes**.

🧠 MEMORIZE

mAP@0.5 (mAP50): a box counts as correct if IoU ≥ 0.5. Lenient: "did you find it roughly?"
mAP@0.5:0.95 (mAP50-95): average of mAP at IoU 0.50, 0.55, ..., 0.95 (10 thresholds). Strict: rewards precise boxes. The main COCO metric.

3 Reading YOLO's Results ★★ IMPORTANT

Column	Meaning
Images / Instances	validation images / labeled objects per class
Box(P), R	precision, recall at the chosen confidence
mAP50	mAP at IoU 0.5
mAP50-95	mAP averaged over IoU 0.5 to 0.95

- Look at the **per-class rows**: a class with low AP needs more / better labeled data.
- **Confusion matrix** has a "background" row/column: background false positives (boxes on nothing) and missed objects.
- **Loss curves** (box loss, class loss) for train and val should go down together, as in DL-05.
- Always **look at predicted images**, not just numbers.

4 Speed Metrics ★★ IMPORTANT

- **Latency:** ms per image (preprocess + inference + postprocess).
- **FPS** = 1000 / latency in ms. Real-time video ≈ 15 to 30+ FPS.
- Measure on the **target device** (your Raspberry Pi), not on Colab's GPU.
- Speed knobs: smaller model (n), smaller `imgsz` (640 → 320), export format (NCNN/ONNX), fewer classes doesn't matter much.

metrics in Python

```
metrics = model.val()        # model = YOLO("best.pt")
print(metrics.box.map50)     # mAP@0.5
print(metrics.box.map)       # mAP@0.5:0.95
print(metrics.box.mp, metrics.box.mr)  # mean P, R
print(metrics.box.maps)      # mAP50-95 per class
```

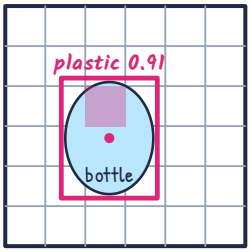
🛠️ **For your project:** Report mAP50, mAP50-95, per-class AP, confusion matrix and FPS on the Pi. Compare at least two model sizes (n vs s) as a speed vs accuracy table.

🎤 **Interview:** "mAP50 vs mAP50-95?" mAP50 only checks rough localization (IoU ≥ 0.5). mAP50-95 averages over stricter IoU thresholds, so it also rewards tight, precise boxes. It's always lower than mAP50.

1 YOLO = You Only Look Once ★★★ MUST KNOW

YOLO looks at the image **once** (one forward pass of a CNN) and directly outputs all boxes, classes and confidences. That's why it is fast enough for real-time video.

- The image is resized (e.g. **640×640**) and processed on a grid of locations at several scales.
- For each location the model predicts: **box** (center, width, height), **class scores**, and how confident it is that an object is there.
- Low-confidence boxes are filtered out (and duplicates removed by NMS in older versions).



1 image → 1 forward pass

grid cell with the object's center predicts:

box (x, y, w, h)
+ confidence
+ class scores

2 Architecture: Backbone → Neck → Head ★★ IMPORTANT

Image 640×640 → **Backbone (CNN)** extracts features → **Neck (FPN / PAN)** mixes small + large scales → **Head** boxes + classes at 3 scales → detections

Backbone: a CNN (like the ones in Part 3) turning pixels into feature maps. Pretrained on large datasets.

Neck: combines detailed early features (good for **small** objects) with deep features (good for **large** objects).

Head: predicts boxes and classes. **Loss** = box loss (IoU-based) + classification loss (BCE), summed.

3 The YOLO Family ★ GOOD TO KNOW

Year	Version	Note
2016	YOLOv1	Joseph Redmon; first real-time single-pass detector
2018	YOLOv3	predictions at 3 scales
2020	YOLOv5	Ultralytics, PyTorch, easy to train
2023	YOLOv8	anchor-free, detect / segment / pose / classify
2024	YOLO11	more accurate, fewer parameters
2026	**YOLO26**	NMS-free end-to-end, built for CPUs and edge devices

Version numbers move fast. Check docs.ultralytics.com for the newest model before starting.

4 Model Sizes ★★★ MUST KNOW

Size	Speed	Accuracy	Use for
n (nano)	fastest	lowest	Raspberry Pi, phones, CPU (YOLO26n ≈ 2.4M params)
s (small)	fast	good	good balance
m (medium)	medium	better	laptop / GPU
l, **x**	slow	best	strong GPU, offline analysis

🧠 MEMORIZE

Weights file names: `yolo26n.pt` (detect) · `yolo26n-seg.pt` (segment) · `yolo26n-cls.pt` (classify) · `yolo26n-pose.pt` · `yolo26n-obb.pt` (rotated boxes). Swap **n** for s / m / l / x.

YOLO's built-in tricks

- **Mosaic augmentation:** 4 images stitched into one → more objects and contexts per image.
- Flips, scaling, HSV color shifts, translation, all applied with boxes adjusted automatically.
- Pretrained on COCO (80 everyday classes, including bottle and cup) → transfer learning by default.

🎤 **Interview:** "Why is YOLO fast?" It's a one-stage detector: one CNN pass predicts all boxes and classes together, with no separate region-proposal step.

train() setting	Meaning (default)
`epochs`	passes over the data (100)
`imgsz`	training image size (640); bigger helps small objects
`batch`	images per step (16); lower it if out of memory, −1 = auto
`patience`	early-stop after N epochs without improvement (100)
`device`	0 = first GPU, "cpu", or "mps" (Mac)
`close_mosaic`	turn mosaic off for the last N epochs (10)

Reading the training log

- **box_loss**: how wrong box positions/sizes are.
- **cls_loss**: how wrong class predictions are (plus a third loss term, depending on version).
- **Instances**: objects in the current batch; **Size**: image size.
- After each epoch: P, R, mAP50, mAP50-95 on the val set. **best.pt** = highest val fitness, **last.pt** = final epoch.

1 Data: the Most Important Part ★★★ MUST KNOW

1. **Collect** varied photos: different lighting, backgrounds, angles, distances, clean and dirty items, single items and **piles**. Include images with **no waste** too (background images reduce false positives).
2. **Label** boxes with **Roboflow**, **CVAT** or **Label Studio** → export in **YOLO format**. Box every visible object, tight to its edges, consistently.
3. **Split** ~70 / 20 / 10 (train / val / test) by scene or location, not randomly, so near-duplicate photos don't leak across splits.
4. **Check balance:** count instances per class. Collect more for rare or weak classes.

Folder structure

```
datasets/waste/
├── images/
│   ├── train/ img001.jpg ...
│   ├── val/
│   └── test/
└── labels/
    ├── train/ img001.txt ...
    ├── val/
    └── test/
```

🧠 MEMORIZE

Label file (same name as the image, one line per object):

`class cx cy w h` (normalized 0 to 1)

```
0 0.3125 0.5000 0.3125 0.5000
3 0.7100 0.6200 0.1500 0.2200
```

No objects → empty .txt file.

2 data.yaml 🧠 MEMORIZE

data.yaml

```yaml
path: datasets/waste      # dataset root
train: images/train       # relative to path
val: images/val
test: images/test         # optional
names:
  0: plastic
  1: paper
  2: metal
  3: glass
  4: cardboard
  5: organic
```

Labels are found automatically by replacing `images` with `labels` in each path.

3 Train, Evaluate, Predict ★★★ MUST KNOW

yolo_waste.py (pip install ultralytics)

```python
from ultralytics import YOLO

model = YOLO("yolo26n.pt")              # pretrained nano model
model.train(data="data.yaml", epochs=100, imgsz=640,
            batch=16, patience=20)      # stop if no gain
# saved to runs/detect/train/weights/best.pt
# (later runs: train2, train3... check the printed path)

model = YOLO("runs/detect/train/weights/best.pt")
metrics = model.val(split="test")       # final, once
print(metrics.box.map50, metrics.box.map)

results = model.predict("pile.jpg", conf=0.25, save=True)
for r in results:
    for box in r.boxes:
        name = r.names[int(box.cls)]
        x1, y1, x2, y2 = box.xyxy[0].tolist()
        print(name, round(float(box.conf), 2),
              [round(v) for v in (x1, y1, x2, y2)])
```

4 Deploy on a Raspberry Pi ★★ IMPORTANT

export + webcam

```python
model.export(format="ncnn")   # also "onnx", "tflite"
# may ask for extra packages, e.g. pip install onnx
pi_model = YOLO("runs/detect/train/weights/best_ncnn_model")
pi_model.predict(source=0, imgsz=320, show=True)  # 0 = webcam
```

- Use the **nano** model and a smaller `imgsz` (320 to 416) for speed; check accuracy doesn't drop too much.
- Measure FPS on the Pi. Then connect the prediction to a servo/bin with GPIO.

same thing from the terminal (CLI)

```
yolo detect train data=data.yaml model=yolo26n.pt \
    epochs=100 imgsz=640
yolo detect predict model=best.pt source=pile.jpg conf=0.25
```

Improve your mAP (in order)

1. **Fix labels**: missing boxes and wrong classes hurt more than anything.
2. **More data** for the weakest classes (check per-class AP).
3. Train longer (more epochs) if val mAP is still rising.
4. Bigger model (n → s) or bigger `imgsz` for small objects.
5. Add background images if you get boxes on nothing.

🛠️ **For your project:** Strong report = local Nepali waste images + per-class results + a test showing how a model trained on public data performs on your local photos + FPS on the Pi.

⚠️ **Common mistake:** Evaluating on the val set you used for early stopping and calling it the final score. Keep a separate test split.

1 Every Project, Same Checklist ★★★ MUST KNOW

1. **Define** the task, the metric that matters, and a target (e.g. "recall ≥ 0.9 on attacks").
2. **Get data**, look at it, note its source and license.
3. **Split first** (stratified; by scene for images). Lock the test set.
4. **Baseline**: the simplest model that works. Record its score.
5. **Improve** one change at a time; log every experiment.
6. **Tune** with CV / validation, never with the test set.
7. **Final test** once. Report several metrics + confusion matrix.
8. **Error analysis**: look at the wrong predictions. What do they have in common?
9. **Package**: clean code, requirements.txt, saved model, README.
10. **Show it**: demo image/GIF/video, short write-up, push to GitHub.

Experiment log (keep it in a table)

#	Change	Val metric	Note
1	baseline LogReg	F1 0.71	underfits
2	Random Forest	F1 0.84	better
3	+ class_weight	F1 0.87	rare class recall ↑

Reproducibility

- Fix seeds: `random_state=42`, `torch.manual_seed(42)`.
- Save versions: `pip freeze > requirements.txt`.
- Save the exact split (file lists) and the best model.
- Run important experiments with 3 to 5 seeds; report mean ± std.

2 A README That Gets You Interviews ★★ IMPORTANT

README.md structure

```
# Waste Detection with YOLO26 on Raspberry Pi
One-line summary + demo GIF / image

## Problem      why it matters (waste in Nepal)
## Dataset      source, images, classes, split
## Method       model, settings, augmentation
## Results      mAP50, mAP50-95, per-class, FPS
## Limitations and next steps
## How to run   install, train, predict
## Project structure
```

💡 **Tip:** Recruiters spend about a minute on a repo. Put the **demo image** and the **results table** at the top.

💡 **Tip:** Pin your 2 best project repos on your GitHub profile; keep the AI Udaan repo as your learning log.

3 Top Mistakes and Fixes ★★★ MUST KNOW

Mistake	Fix	Page
Training or fitting a scaler on test data (leakage)	split first; fit on train; use Pipelines	ML-02, ML-17
Reporting accuracy on imbalanced data	recall, F1, macro-F1, confusion matrix, PR-AUC	ML-10
Tiny test set → "100% accuracy"	bigger data, cross-validation, mean ± std	ML-11
Not scaling for KNN / SVM / gradient descent / NN	StandardScaler inside the pipeline	ML-03
Predicting raw x with a polynomial model	Pipeline with PolynomialFeatures	ML-06
softmax / sigmoid before CrossEntropyLoss / BCEWithLogitsLoss	output raw logits	DL-02, DL-03
Forgetting `zero_grad()` / `model.eval()` / `no_grad()`	use the DL-07 template every time	DL-07
Different preprocessing at train vs test time	same Resize / Normalize; save the transforms	CV-05
Near-duplicate images across splits	split by scene / location / date	CV-09
Messy or missing box labels	review labels before training more	CV-09
Changing many things at once	one change per experiment, log it	P-01

🛠️ **For your project: Intrusion detection (missing data study):** ML-02 (splits, leakage) → ML-03 (imputation) → ML-10 (metrics) → ML-11 (CV, imbalance) → ML-14 (RF, XGBoost) → ML-17 (pipeline per imputer).

🛠️ **For your project: Waste detection:** CV-01 to CV-03 (CNN basics) → CV-05 (transfer learning) → CV-06, CV-07 (boxes, metrics) → CV-08, CV-09 (YOLO, data, Raspberry Pi).

> Answer in 3 parts: **what it is → why it matters → an example from your project.**

1. Overfitting vs underfitting? Overfit: great on train, poor on new data (memorized noise). Underfit: poor on both (too simple). Diagnose with train vs val error.

2. Bias-variance trade-off? More complexity lowers bias but raises variance. Aim for the lowest total validation error.

3. How do you prevent overfitting? More data/augmentation, regularization (L1/L2, dropout), simpler model, early stopping, cross-validation.

4. L1 vs L2? L1 (Lasso) adds $\Sigma|w|$, drives some weights to 0 (feature selection). L2 (Ridge) adds Σw^2, shrinks all weights smoothly.

5. Precision vs recall? Precision: of predicted positives, how many are right. Recall: of real positives, how many found. IDS/medical → recall; spam → precision.

6. What is F1, and when use it? Harmonic mean of precision and recall; use for imbalanced data or when both errors matter.

7. ROC-AUC? Area under TPR-vs-FPR curve across thresholds; probability the model ranks a random positive above a random negative. 0.5 = random.

8. Handling imbalanced data? Right metrics, stratified splits, class weights, threshold tuning, SMOTE on training folds only, more minority data.

9. Why a validation set and a test set? Validation guides choices (tuning); the test set gives an unbiased final score because it was never used to decide anything.

10. What is data leakage? Info from test/future data entering training (e.g. scaling before splitting), giving fake high scores. Prevent with split-first + pipelines.

11. Why scale features? Distance-based models and gradient descent are dominated by large-valued features otherwise. Trees don't need it.

12. Bagging vs boosting? Bagging: parallel models on bootstrap samples, averages → lowers variance (RF). Boosting: sequential models fixing previous errors → lowers bias (XGBoost).

13. How does a decision tree choose splits? It picks the feature/threshold with the largest information gain (entropy drop) or Gini decrease.

14. K-Means: how to choose k? Elbow on inertia, silhouette score, domain knowledge. Scale features first.

15. Batch vs mini-batch vs SGD? All data vs a small batch vs one sample per update. Mini-batch balances speed, stability and GPU use.

16. What is backpropagation? Using the chain rule to compute the loss gradient for every weight, from the output layer backwards; then gradient descent updates them.

17. Vanishing gradients and fixes? Gradients shrink through many layers (sigmoid/tanh). Fix: ReLU, He init, BatchNorm, residual connections.

18. Sigmoid vs softmax? Sigmoid: independent probability per output (binary, multi-label). Softmax: probabilities that sum to 1 (one class out of many).

19. Dropout and BatchNorm at test time? Dropout is off; BatchNorm uses running mean/variance. That's why `model.eval()` is needed.

20. Why CNNs for images? Local filters with shared weights: few parameters, keep spatial structure, detect patterns anywhere.

21. What does pooling do? Downsamples feature maps (e.g. 2×2 max halves size), cuts compute, adds small shift tolerance; no parameters.

22. Transfer learning? Reuse a model pretrained on a large dataset; replace the head; train it, then fine-tune with a small lr. Great for small datasets.

23. IoU, NMS, mAP? IoU: box overlap ratio. NMS: removes duplicate boxes. mAP: mean over classes of the area under the precision-recall curve.

24. Why is YOLO fast? One-stage: a single CNN pass predicts all boxes and classes; no region-proposal stage. YOLO26 also removes NMS.

25. Parameters vs hyperparameters? Parameters (weights, biases) are learned; hyperparameters (lr, depth, k, batch size) are set by you and tuned on validation data.

26. Your model has 99% train and 70% val accuracy. What next? It's overfitting: add data/augmentation, regularize, simplify, early stop; check for label noise.

"Tell me about your project" (2-minute structure)

1. **Problem:** "Waste isn't sorted at the source in Nepal, so I built a detector for 6 waste types."
2. **Data:** where it came from, how many images, how you labeled and split it (by scene, to avoid leakage).
3. **Model & why:** "YOLO26n, because it's real-time on a Raspberry Pi."
4. **Results:** mAP50, mAP50-95, the weakest class and why, FPS on the device.
5. **What you'd improve** next. Showing you understand the limits impresses interviewers.

Formulas You Should Know by Heart

MEMORIZE

MEMORIZE

Regression

$\hat{y} = w \cdot x + b$
$MSE = (1/n)\Sigma(y - \hat{y})^2$
$MAE = (1/n)\Sigma|y - \hat{y}|$
$RMSE = \sqrt{MSE}$
$R^2 = 1 - SS_{res}/SS_{tot}$
$\beta = (X^TX)^{-1}X^Ty$

MEMORIZE

Classification

$\sigma(z) = 1/(1 + e^{-z})$
$BCE = -mean[y \cdot \log p + (1-y) \cdot \log(1-p)]$
$softmax = e^{z_i}/\Sigma e^{z_j}$
$CE = -\log(p_{true})$
Gradient: $X^T(p - y)/n$

MEMORIZE

Metrics

Precision = TP/(TP+FP)
Recall = TP/(TP+FN)
F1 = 2PR/(P+R)
Accuracy = (TP+TN)/all
FPR = FP/(FP+TN)

MEMORIZE

Learning

$w \leftarrow w - \alpha \cdot \partial L/\partial w$
Chain rule: $\partial L/\partial w = \partial L/\partial a \cdot \partial a/\partial z \cdot \partial z/\partial w$
Iterations/epoch = $\lceil N / batch \rceil$
Adam default lr = 1e-3
Initial CE loss $\approx \ln(C)$

MEMORIZE

Trees & data

Entropy = $-\Sigma\ p \log_2 p$
Gini = $1 - \Sigma p^2$
IG = $H(parent) - \Sigma(n_k/n)H(child_k)$
z-score = $(x - \mu)/\sigma$
IQR rule: $Q1 - 1.5 \cdot IQR$, $Q3 + 1.5 \cdot IQR$

MEMORIZE

CNN & detection

Out = $\lfloor (W - K + 2P)/S \rfloor + 1$
Conv params = $(K \cdot K \cdot C_{in} + 1) \cdot C_{out}$
Linear params = in·out + out
IoU = overlap / union
ResNet: $y = F(x) + x$

Which Output + Loss?

MEMORIZE

Task	Last layer	Loss (PyTorch)	Prediction	Main metric
Regression	1, no activation	MSELoss	value	RMSE, MAE, R^2
Binary	1 logit	BCEWithLogitsLoss	sigmoid ≥ 0.5	F1, recall, ROC-AUC
Multi-class	C logits	CrossEntropyLoss	argmax	accuracy, macro-F1
Multi-label	C logits	BCEWithLogitsLoss	each sigmoid ≥ 0.5	micro/macro F1, hamming
Detection	YOLO head	box + class loss	boxes ≥ conf	mAP50, mAP50-95

Code Skeletons

MEMORIZE THE STEPS

scikit-learn

```
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
pipe = make_pipeline(StandardScaler(), RandomForestClassifier())
scores = cross_val_score(pipe, X_tr, y_tr, cv=5, scoring="f1")
pipe.fit(X_tr, y_tr)
print(classification_report(y_te, pipe.predict(X_te)))
```

PyTorch train + eval

```
for epoch in range(EPOCHS):
    model.train()
    for x, y in train_dl:
        x, y = x.to(device), y.to(device)
        optimizer.zero_grad()
        loss = criterion(model(x), y)
        loss.backward()
        optimizer.step()
    model.eval()
    with torch.no_grad():
        preds = model(x_val.to(device)).argmax(dim=1)
```

transfer learning

```
model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
for p in model.parameters():
    p.requires_grad = False
model.fc = nn.Linear(model.fc.in_features, num_classes)
optimizer = torch.optim.AdamW(model.fc.parameters(), lr=1e-3)
```

YOLO

```
model = YOLO("yolo26n.pt")
model.train(data="data.yaml", epochs=100, imgsz=640)
metrics = model.val()        # metrics.box.map50, .map
model.predict("img.jpg", conf=0.25, save=True)
model.export(format="ncnn")  # or "onnx" for deployment
```

After this handbook (when your projects are done)

Deploy: serve a model with FastAPI, package with Docker.

NLP: tokenization, embeddings, transformers, fine-tuning (Hugging Face).

Segmentation: YOLO-seg, U-Net for pixel masks.

Research: read 1 paper a week, reproduce one result, write it up.

"You made it to the end. Now build: one tabular project, one CNN, one YOLO project. Understanding comes from doing." 🚀

Learn · Code · Build · Repeat